Java Fundamentals: Complete e-Content for Beginners

Dr. A. Bamila Virgin Louis

Asso.Prof/ CSE

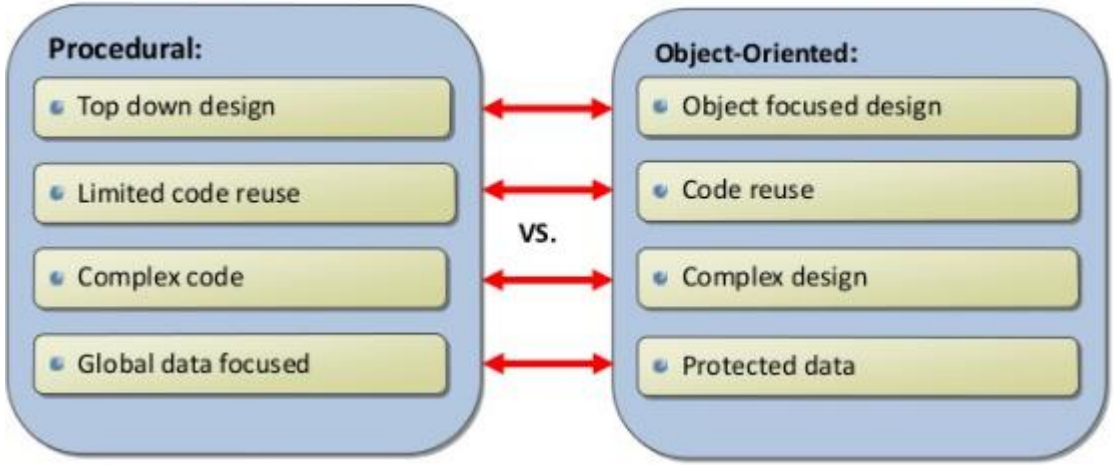
SXCCE

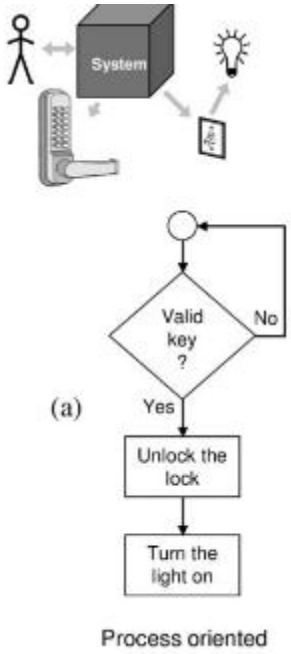
INTRODUCTION TO OOP AND JAVA FUNDAMENTALS

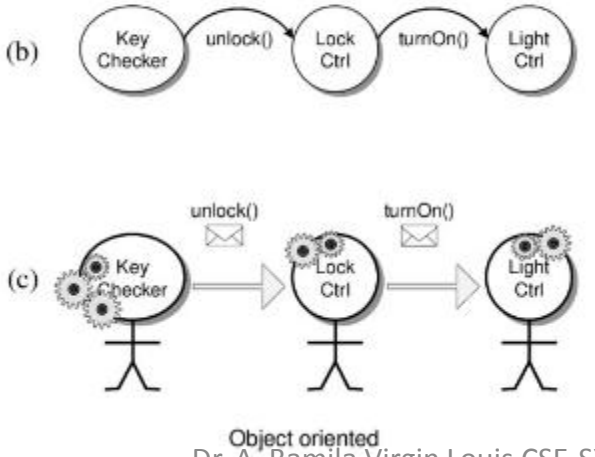
Object Oriented Programming - Abstraction – objects and classes - Encapsulation- Inheritance - Polymorphism- OOP in Java – Characteristics of Java – The Java Environment - Java Source File - Structure – Compilation. Fundamental Programming Structures in Java – Defining classes in Java – constructors, methods -access specifiers - static members -Comments, Data Types, Variables, Operators, Control Flow, Arrays , Packages - JavaDoc comments.

Two Paradigms

- Process-oriented model
- Object-oriented model



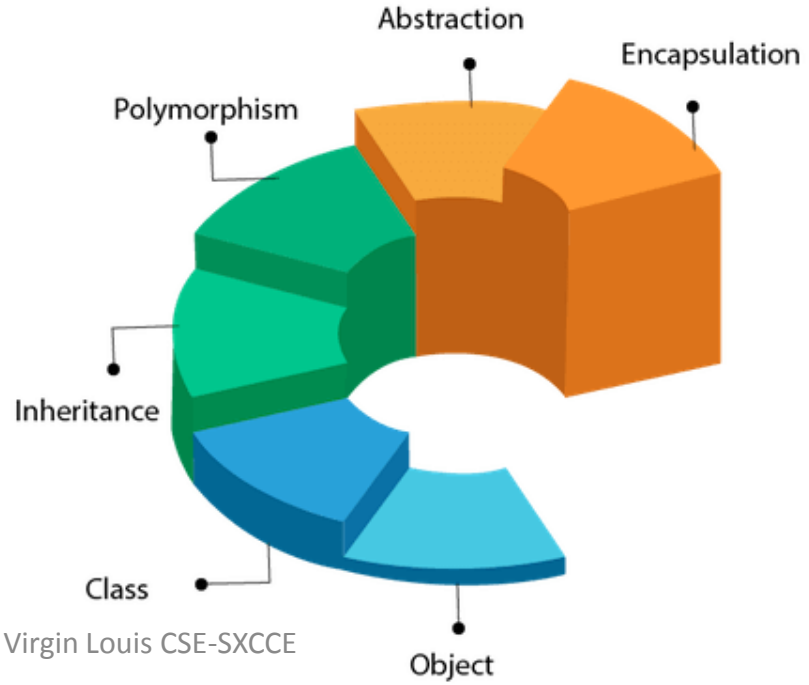




Object Oriented Programming

- Object-Oriented Programming is a paradigm that provides many concepts, such as **inheritance**, **data binding**, **polymorphism**, etc.
- The popular object-oriented languages are Java, C#, PHP, Python, C++, etc.

- OOPs (Object-Oriented Programming System)



Object

- **Object** means a real-world entity.
- Properties of Objects- state and behaviour.
- **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects.
- The object paradigm is based on the idea that we can construct software by assembling communities of interacting objects.
- It simplifies software development and maintenance.

Class

- Class- collection of objects.
- Objects are instances of class.
- Classes are user-defined data types.
- A class defines the characteristics (structure and behavior) that are common to a group of similar objects.
  - defines the variables that hold the object's state
  - defines the methods that describe the object's behaviours
- Each object of a given class contains the structure and behavior defined by the class.
- A class can also be defined as a blueprint from which you can create an individual object.

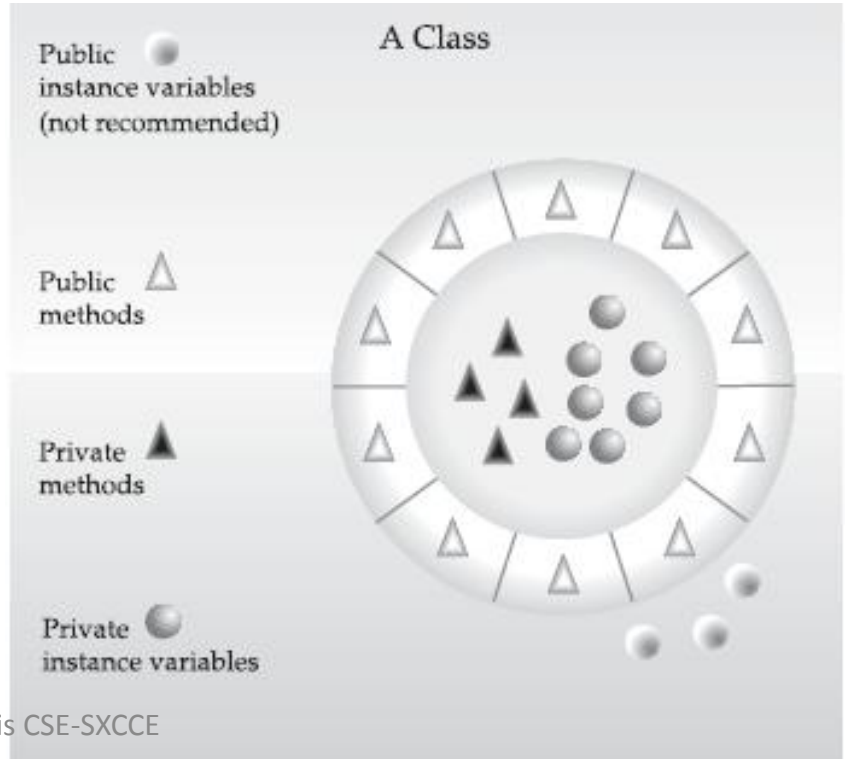
The General Form of a Class

```
class classname
 {
    type instance-variable1;
     // ...
    type instance-variableN;
   type methodname1(parameter-list)
    {
       // body of method
     }
      // ...
   type methodnameN(parameter-list)
    {
       // body of method
     }
}
```

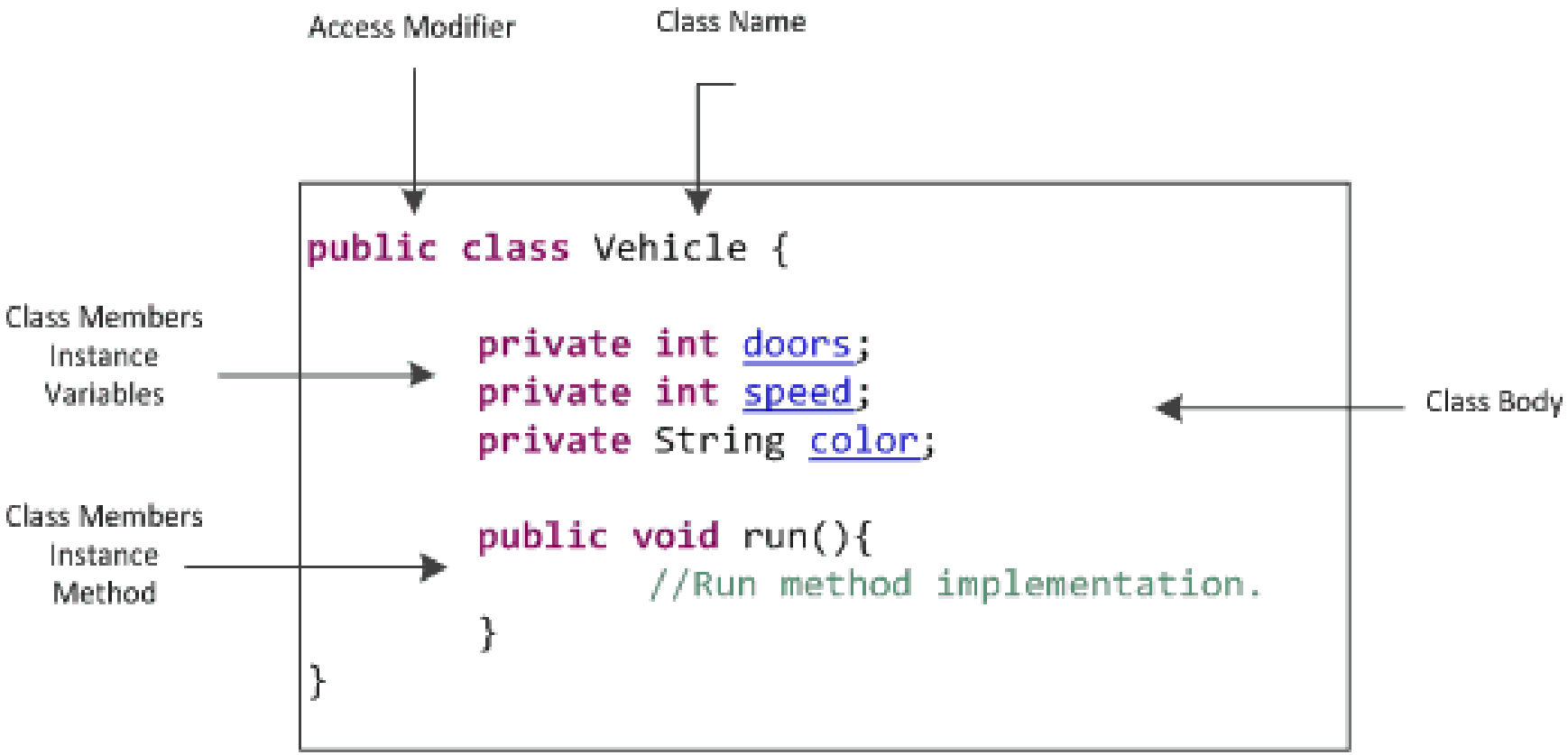
Access modifier

Public

Private

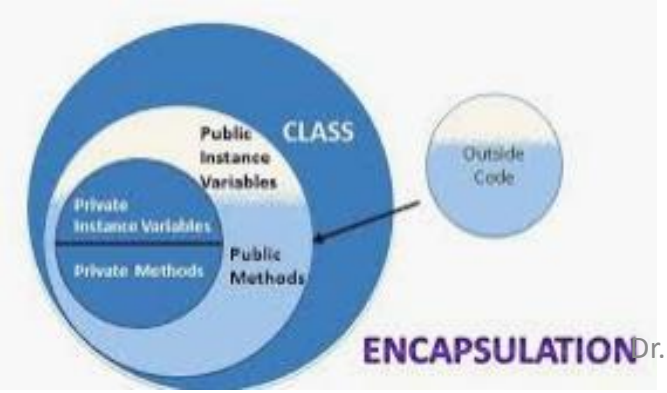


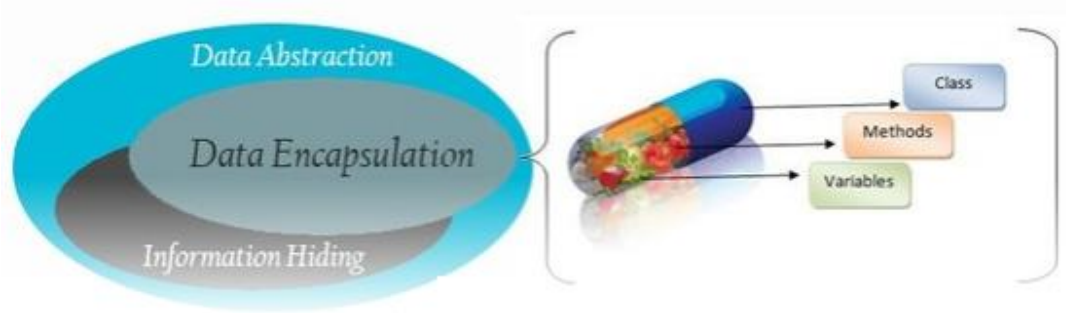
Class


Access Modifier
Class Name
public class Vehicle {
private int doors;
private int speed;
private String color;
public void run(){
//Run method implementation.
}
}
Class Members
Instance
Variables
Class Members
Instance
Method
Class Body

Data Abstraction and Encapsulation

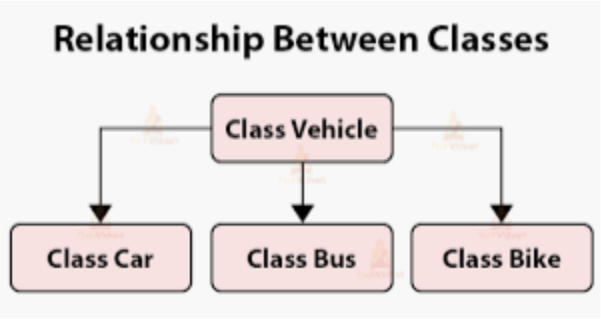
- *Encapsulation:* Binding (or wrapping) code and data together into a single unit. Eg: java class.
- Methods provide the interface between the object's data and the program.
- Data hiding: insulation of the data from direct access by the program.
- Abstraction: Hiding internal details and showing functionality.
  - Examples: TV, Phone call, Driving a car, etc
- Classes use the concept of abstraction, so they are known as abstract data type(ADT).





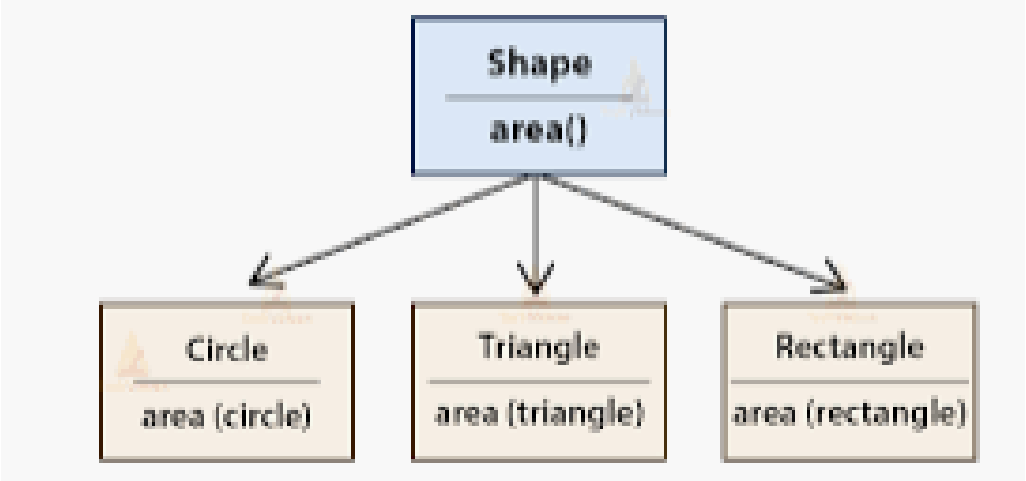
Inheritance

- Objects of one class acquire the properties of objects of another class.
- Inheritance supports the concept of hierarchical classification.
- Inheritance allows to reuse classes by deriving a new class from an existing one.
- The existing class is called the parent class, or superclass, or base class.
- The derived class is called the child class or subclass.
- The child class inherits characteristics of the parent class



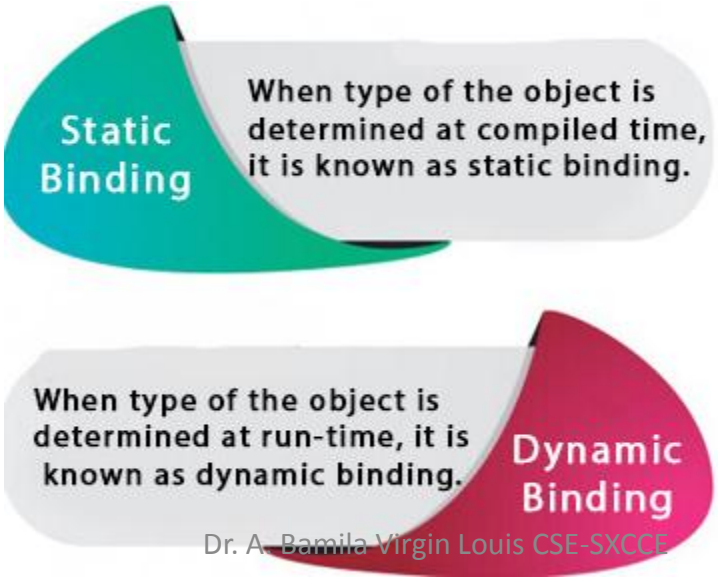
Polymorphism

- Polymorphism is the ability of an object to take on many forms.
- Polymorphism is extensively used in implementing inheritance.
- Objects having different internal structures can share the same external interface.



Dynamic Binding

- Binding: linking of a procedure call to the code to be executed in response to the call.
- Dynamic binding: the code associated with a given procedure call is not known until the time of the call at runtime.
- It is associated with polymorphism and inheritance.
- A procedure call associated with a polymorphic reference depends on the dynamic type of that reference.

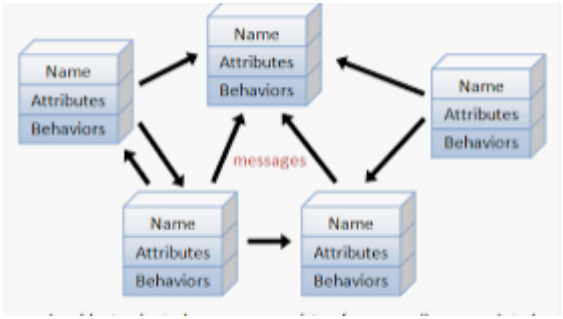


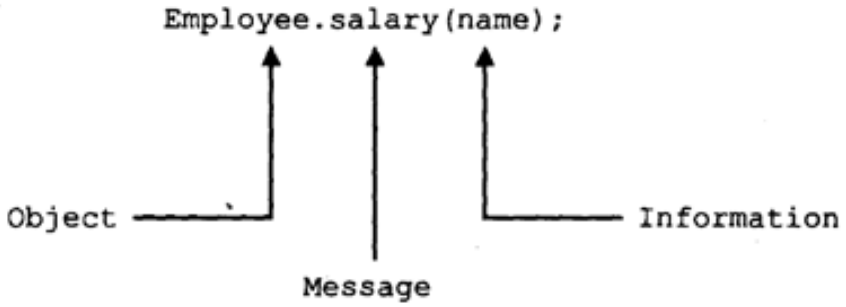
Message Communication

- Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another.
- process of programming in an object-oriented language:

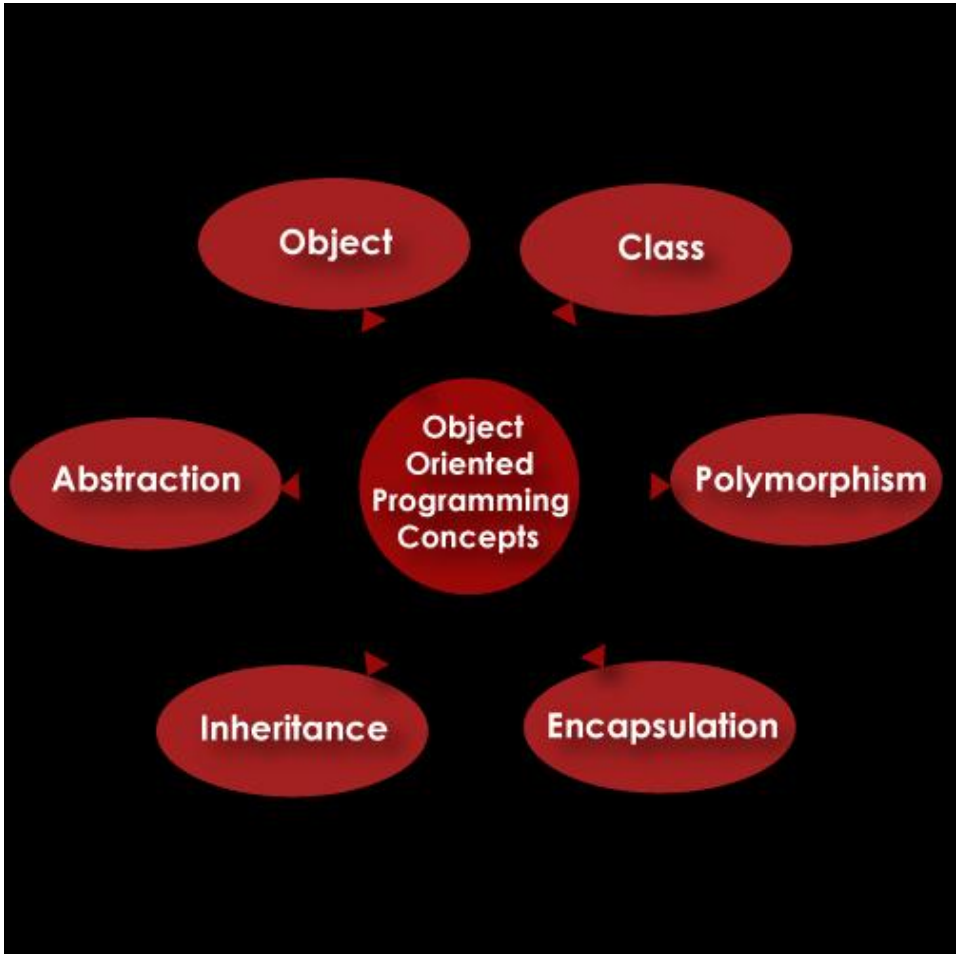
1. Creating classes that define objects and their behaviour.
2. Creating objects from class definitions.
3. Establishing communication among objects.

- A message for an object is a request for execution of a procedure





Summary


Object
Class
Object Oriented Programming Concepts
Abstraction
Polymorphism
Inheritance
Encapsulation

Java

- **James Gosling**, **Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.
- Initially designed for small, embedded systems in electronic appliances like set-top boxes.
- Firstly, it was called **"Greentalk"** by James Gosling, and the file extension was .gt.
- After that, it was called **Oak** and was developed as a part of the Green project.

Java

Why Java named "Oak"?

- **Why Oak?** Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.
- In 1995, Oak was renamed as **"Java"** because it was already a trademark by Oak Technologies.
- Java name was chosen by James Gosling. Java is just a name, not an acronym.

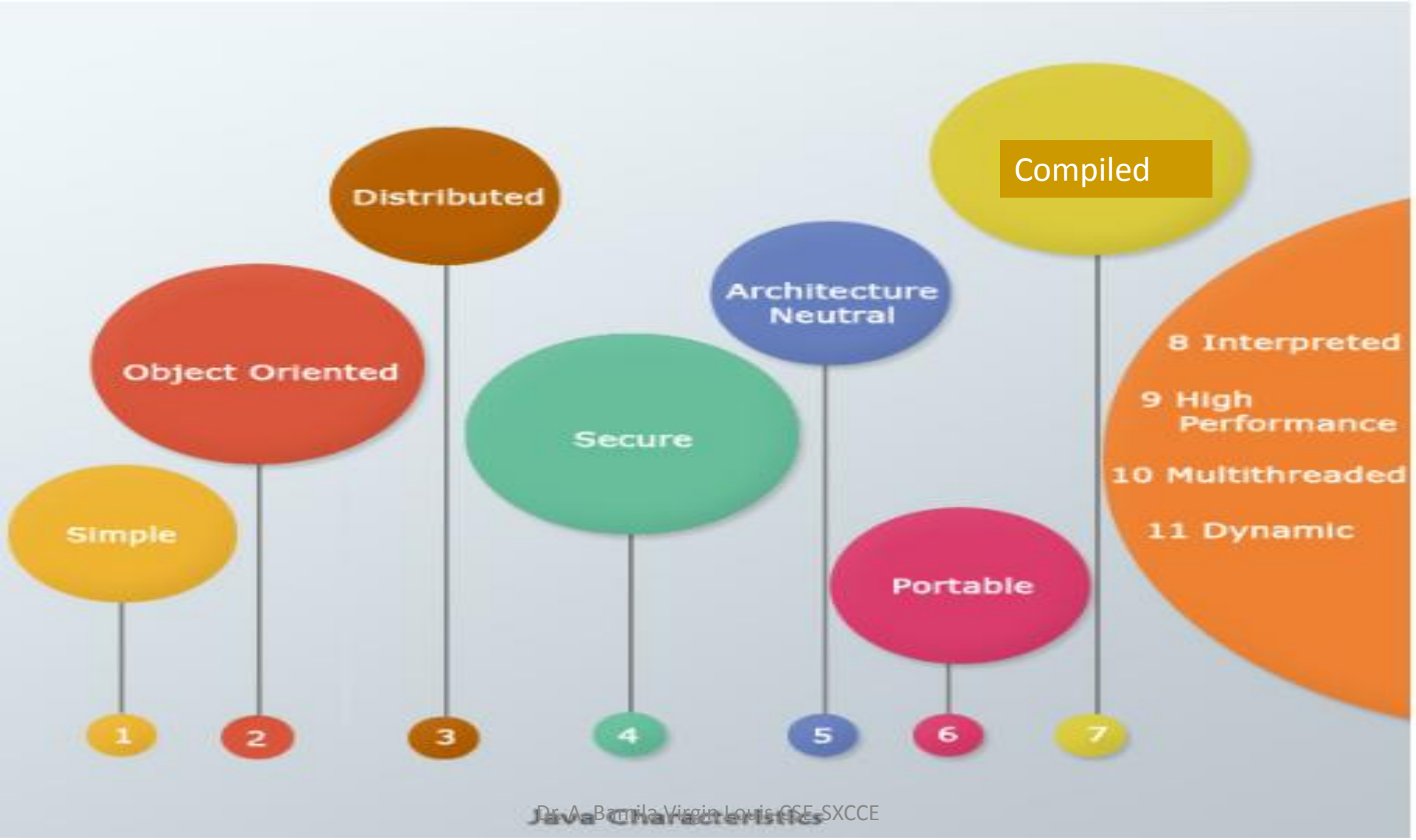
Java

- Java is a robust, general-purpose, high-level programming language and a powerful software platform.
- Object-oriented, distributed, portable and multi-threaded.
- Java follows the *'Write - once - run - anywhere'* approach.

CHARACTERISTICS OF JAVA

- Compiled and Interpreted
- Platform-Independent and Portable
- Object-Oriented
- Robust and Secure
- Distributed
- Familiar, Simple and Small
- Multithreaded and Interactive
- High Performance
- Dynamic and Extensible

CHARACTERISTICS OF JAVA



Differences between Java and C

- Java does not include the C unique statement keywords goto, sizoof, and typedef.
- Java does not contain the data types struct, union and enum .
- Java does not define the type modifiers keywords auto, extern, register, signed, and unsigned.
- Java does not support an explicit pointer type.
- Java does not have a preprocessor and therefore we cannot use # define, # include, and # ifdef statements.
- Java requires that the functions with no arguments must be declared with empty parenthesis and not with the void keyword as done in C.

THE JAVA PROGRAMMING ENVIRONMENT

- All Java programs must run on the Java platform that has two components, the Java Virtual Machine (JVM) and the Java Application Programming Interface (API).

Java Virtual Machine (JVM)

- **Java Virtual Machine (JVM)** is an abstract definition of a computing machine. It is a specification that provides runtime environment in which java bytecode can be executed.
- **Java Runtime Environment (JRE)** is the implementation of the JVM. (interpreter, garbage collector, class loader, java class libraries)
- **Java Development Kit (JDK)** is a set of tools using which Java programs can be developed and compiled.
- **Just In Time Compiler (JIT)** runs in the execution environment to make optimizations on the program.

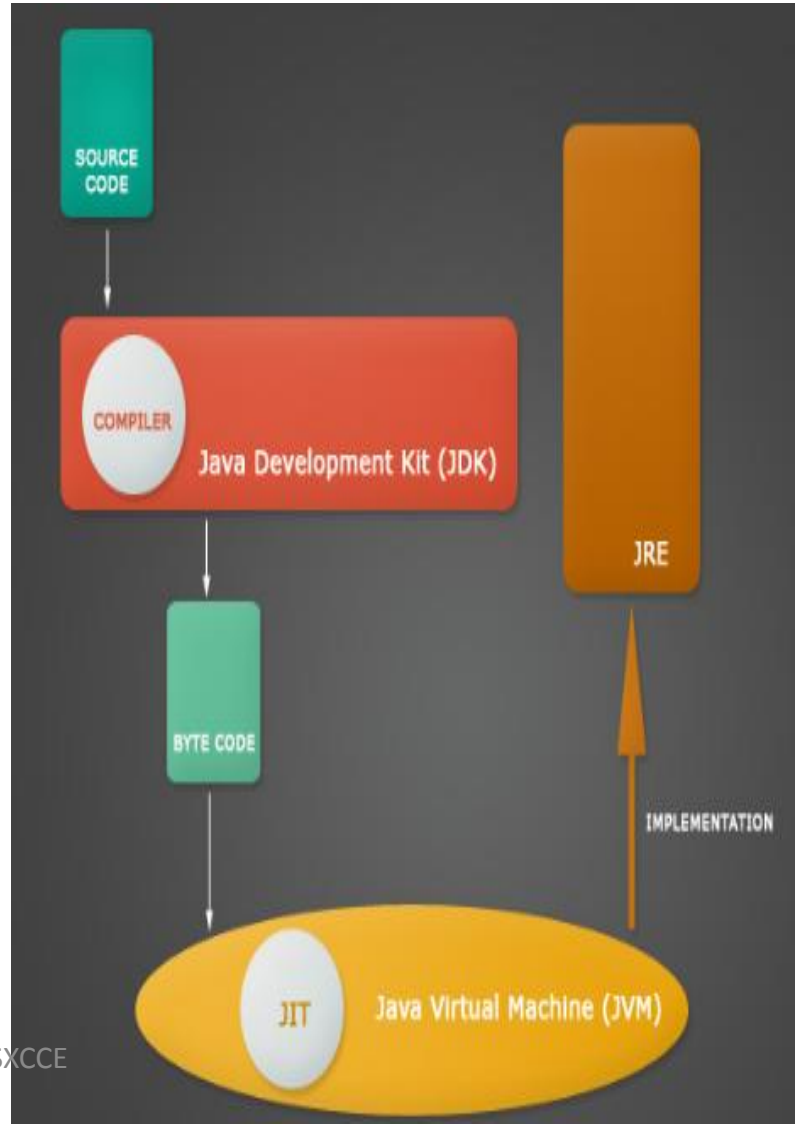
Java Virtual Machine (JVM)

The JVM performs following operation:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment

JVM provides definitions for the:

- Memory area
- Class file format
- Register set
- Garbage-collected heap



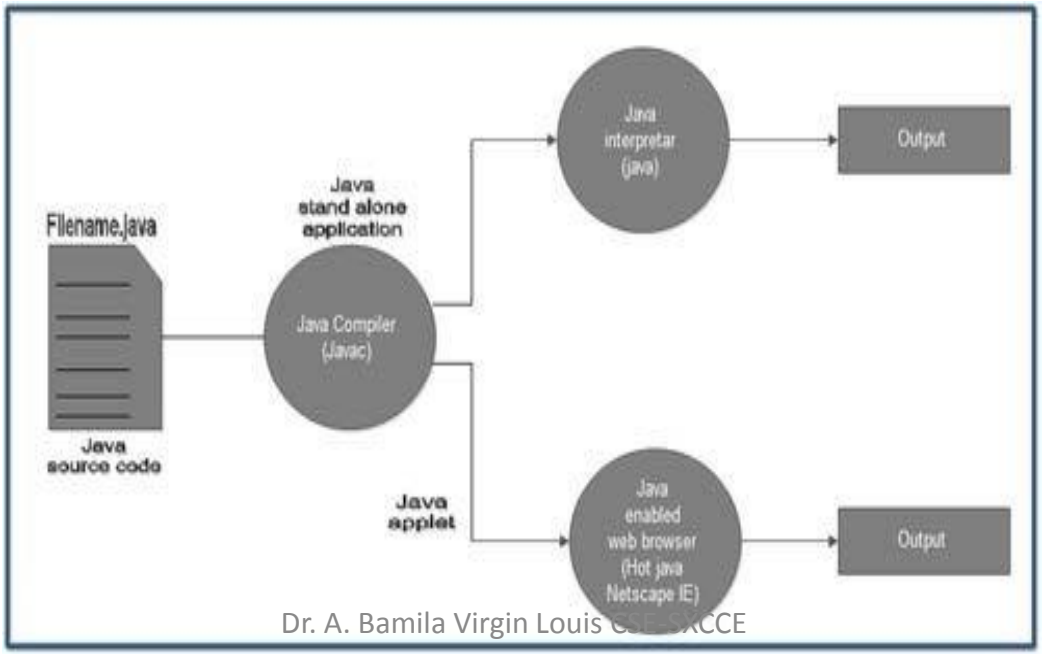
Java API

- With Java API, many types of Java programs can be developed.

➢ Java stand-alone applications- run directly by the Java interpreter

➢ Java Applets - Java applications, embedded inside HTML files

➢ Java Servlets- Java programs, run on a web server.



THE JAVA PROGRAMMING ENVIRONMENT

Steps to prepare the Java development environment:

- Download Java
- Install Java
- Setup Path
- Test installation

To start with Java programming, download Java SE "Standard Edition".

- Downloading the JDK
- Installing the Java Development Kit

www.oracle.com/technetwork/java/javase/downloads

THE JAVA PROGRAMMING ENVIRONMENT

Java Development kit: Collection of tools used for developing and executing java programs.

- appletviewer (for viewing Java applets)
- javac (Java compiler)
- java (Java interpreter)
- javap (Java disassembler)
- javah (for C header files)
- javadoc (for creating HTML documents)
- jdb (Java debugger)

Directory Structure	Description
jdk	(The name may be different, for example, jdk5.0)
bin	The compiler and tools
demo	Look here for demos
docs	Library documentation in HTML format (after expansion of j2sdk*version*-doc.zip)
include	Files for compiling native methods (see Volume II)
jre	Java runtime environment files
lib	Library files
src	The library source (after expanding src.zip)

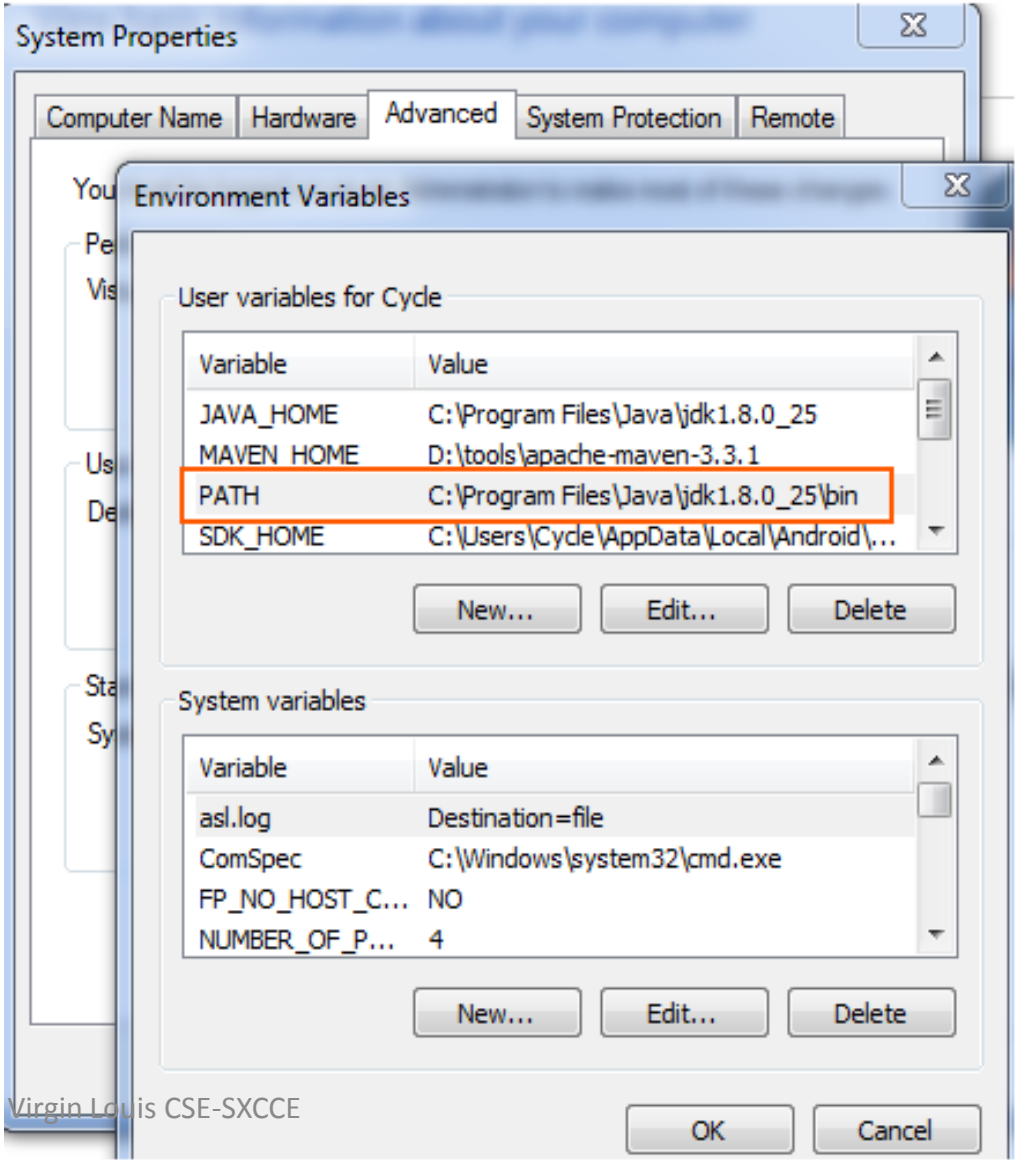
THE JAVA PROGRAMMING ENVIRONMENT

Setting the Executable Path

- Add the jdk/bin directory to the executable path

In Windows,

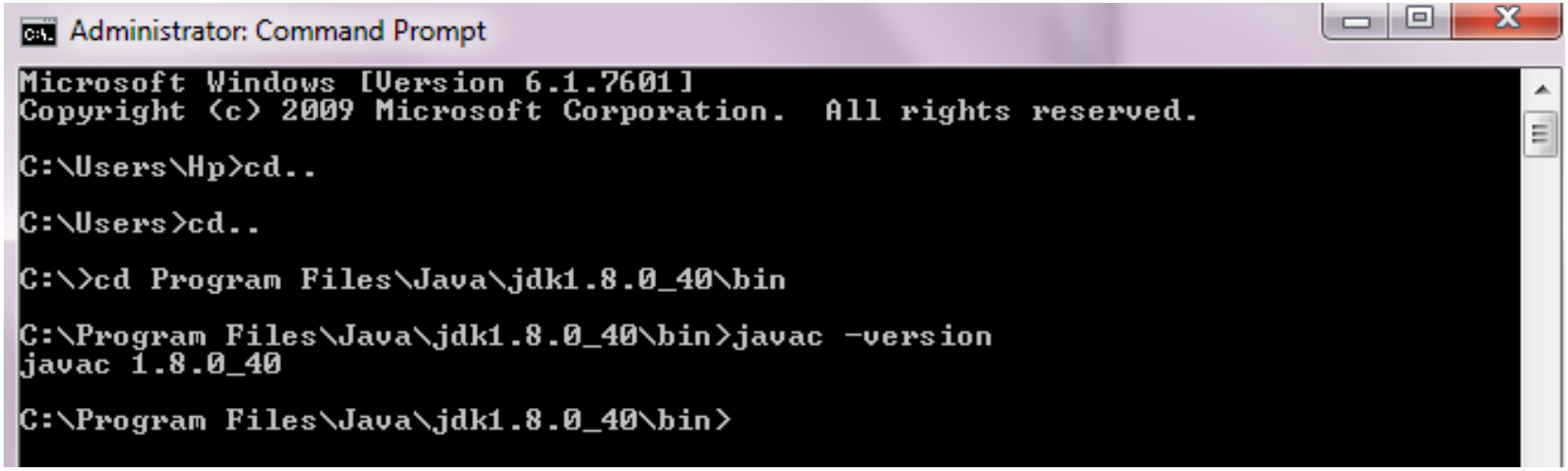
- Go to My computer and select properties, advanced system settings then in System Properties
- Go to 'Advanced' and click 'Environment Variables'



Java Development Kit

Test installation

- To test if the installation is right, open a command prompt and type the command "javac -version". It should respond with the version of the JDK installed. Then you are ready to start writing Java programs.



Choosing a Development Environment

- **Using the Command-Line Tools**

1. Open a shell window.

2. Go to the directory into which you installed the source code.(eg. Welcome.java)

3. Enter the following commands:

```
javac Welcome.java
java Welcome
```

We can see the output in the shell window

- **Using an Integrated Development Environment**

1. Eclipse

2. NetBeans

STRUCTURE OF JAVA PROGRAM

A Java program involves the following sections:

- Documentation Section
- Package Statement
- Import Statements
- Interface Statement
- Class Definition
- Main Method Class
- Main Method Definition

STRUCTURE OF JAVA PROGRAM

Section	Description
Documentation Section	You can write a comment in this section. Comments are beneficial for the programmer because they help them understand the code. These are optional.
Package statement	You can create a package with any name. A package is a group of classes that are defined by a name. That is, if you want to declare many classes within one element, then you can declare it within a package. It is an optional. To create a package, use the statement package packagename;
Import statements	This line indicates that if you want to use a class of another package, then you can do this by importing it directly into your program. Example: import calc.add;

STRUCTURE OF JAVA PROGRAM

Section	Description
Interface statement	Interfaces are like a class that includes a group of method declarations. It's an optional section and can be used when programmers want to implement multiple inheritances within a program.
Class Definition	A Java program may contain several class definitions. Classes are the main and essential elements of any Java program.
Main Method Class	Every Java stand-alone program requires the main method as the starting point of the program. This is an essential part of a Java program. There may be many classes in a Java program, and only one class defines the main method. Methods contain data type declaration and executable statements.

STRUCTURE OF JAVA PROGRAM

ProgramStructure.java

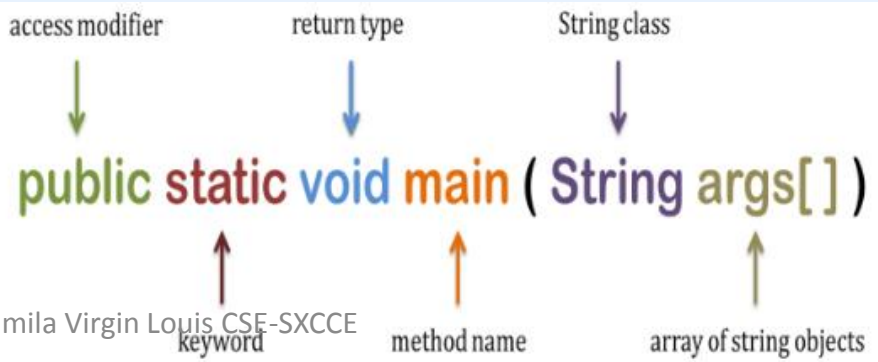
```
1  /**
2   * Javadoc comment in Java. Javadoc comments are used to document code
3   * blah blah blah....
4   */
5
6  package com.testingdocs.javaautomation;
7
8  import com.testingdocs.utilities.SampleClass;
9
10 /**
11  * @author Surendra
12  *
13  */
14
15 public class ProgramStructure {
16     public static void main(String[] args) {
17         // Single line comment, crisp text goes here
18         SampleClass sc = new SampleClass();
19
20     /*
21      This is a multi-line comment in Java. This will split into multiple lines
22      as shown here... SOmething long description goes here....
23     */
24
25     }
26
27 }
```

1

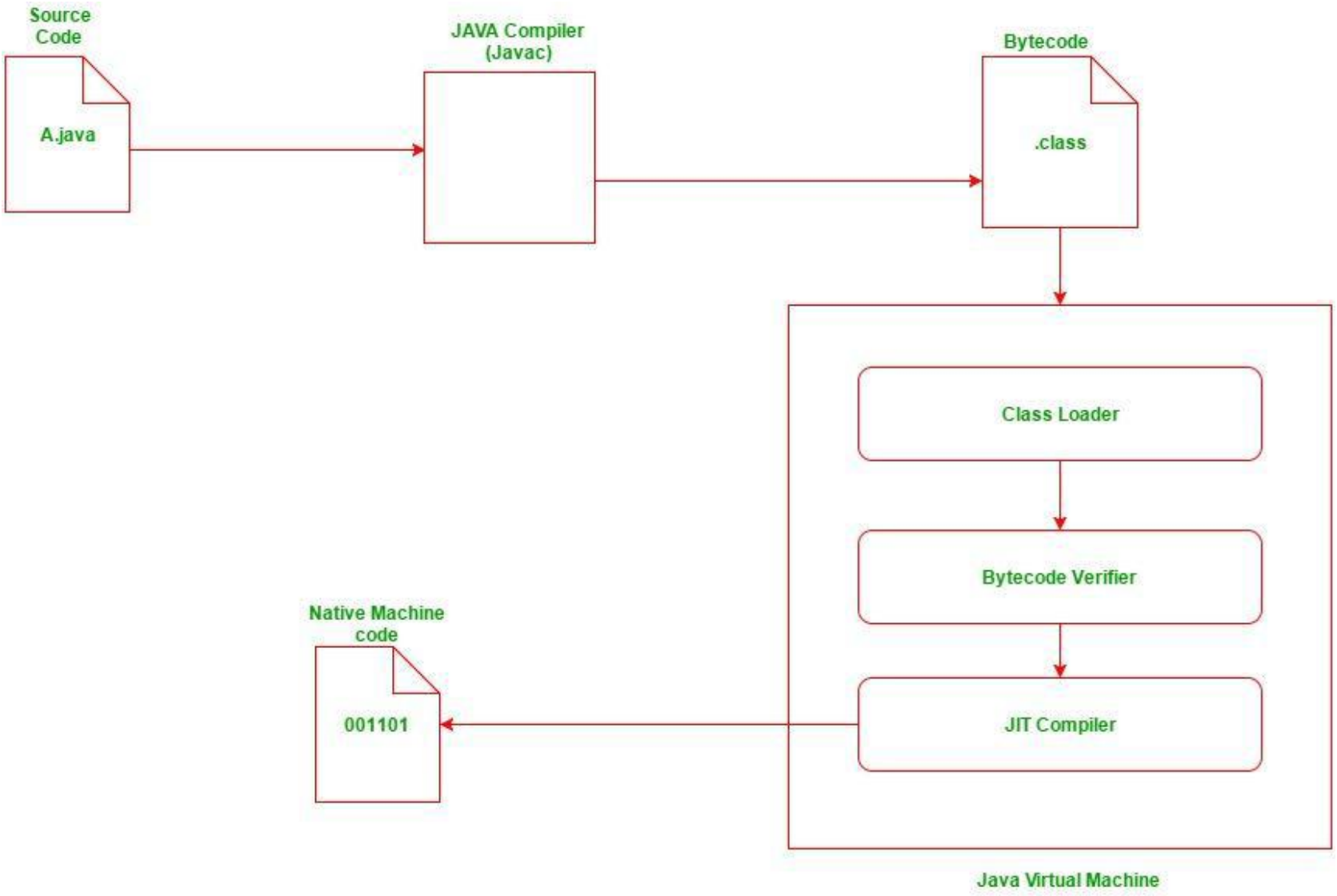
2

3

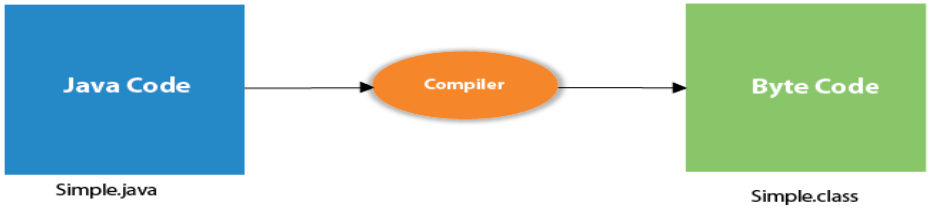
4

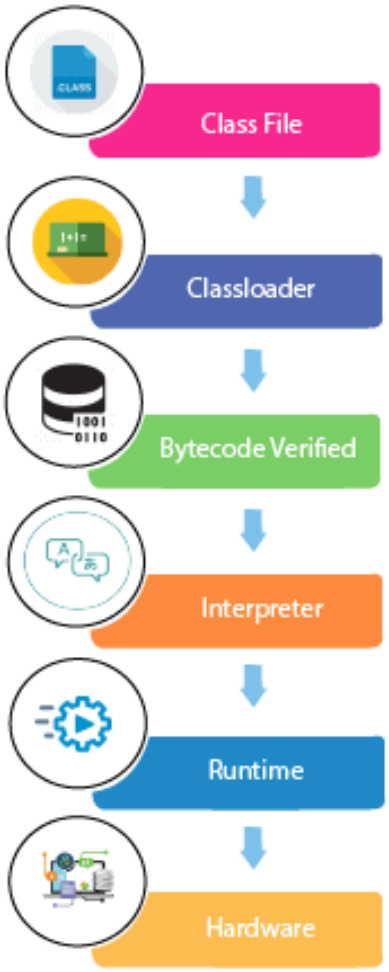


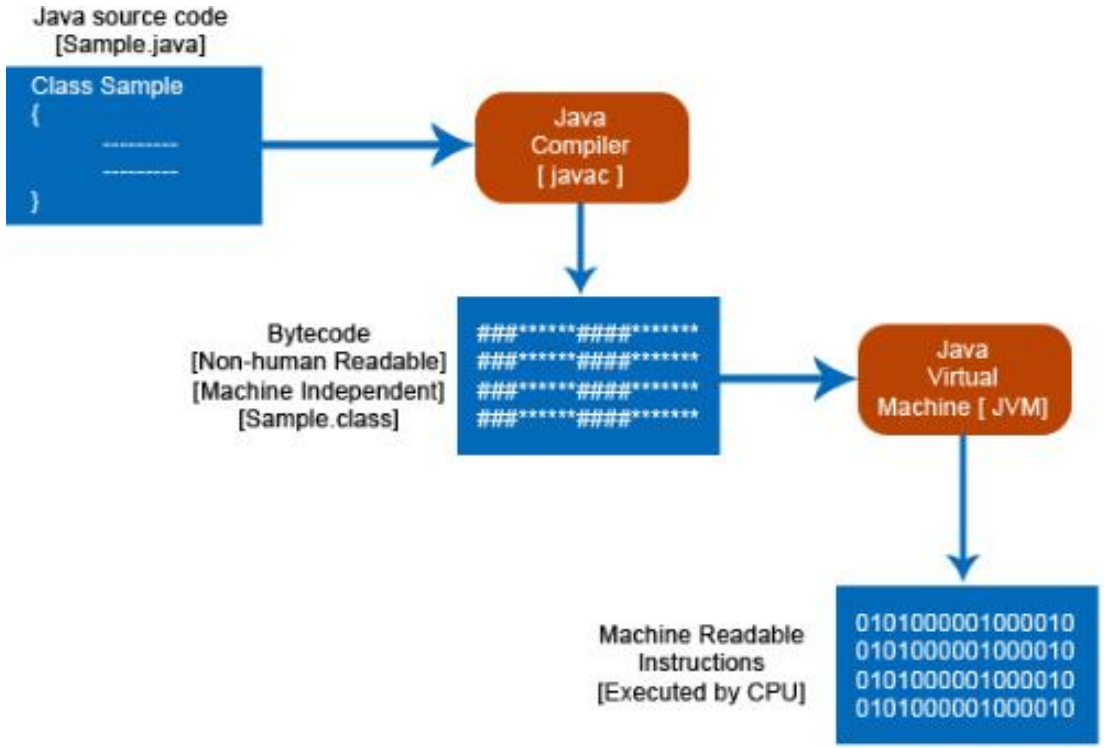
COMPILATION AND EXECUTION OF A JAVA PROGRAM



COMPILATION AND EXECUTION OF A JAVA PROGRAM







FUNDAMENTAL PROGRAMMING STRUCTURES IN JAVA

Comments, Data Types, Variables, Operators, Control Flow, Arrays

COMMENTS

Java has three ways of marking comments.

- The most common form is a //.
- **/* and */**
- **/** to start and a */ to end** comment delimiters are used when longer comments are needed.

```
/**
 * This is my first
 * java program
 */
public class JavaApplication1
{
   public static void main(String[] args)
 {
       System.out.println("welcome");
 }
```

DATA TYPES

The Primitive Types

- Java defines eight primitive types of data: **byte, short, int, long, char, float, double, and boolean**.
- These can be put in four groups:
- **Integers:** This group includes byte, short, int, and long, which are for whole-valued signed numbers.
- **Floating-point numbers:** This group includes float and double, which represent numbers with fractional precision.
- **Characters:** This group includes char, which represents symbols in a character set, like letters and numbers.
- **Boolean:** This group includes boolean, which is a special type for representing true/false values.

Integers

Java defines four integer types: byte, short, int, and long. All of these are signed, positive and negative values.

Name	Width	Range
long	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	–2,147,483,648 to 2,147,483,647
short	16	–32,768 to 32,767
byte	8	–128 to 127

Example

```
// Compute distance light travels using long variables.
class Light
{
public static void main(String args[])
{
int lightspeed;
long days;
long seconds;
long distance;
// approximate speed of light in miles per second
lightspeed = 186000;
days = 1000; // specify number of days here
seconds = days * 24 * 60 * 60; // convert to seconds
distance = lightspeed * seconds; // compute distance
System.out.print("In " + days);
System.out.print(" days light will travel about ");
System.out.println(distance + " miles.");
}
}
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac Light.java

C:\Program Files\Java\jdk1.8.0_40\bin>java Light
In 1000 days light will travel about 160704000000000 miles.
```

Floating-Point Types

- Floating-point numbers, also known as real numbers, are used when evaluating expressions that require fractional precision.
- There are two kinds of floating-point types, float and double, which represent single- and double-precision numbers, respectively.

Name	Width in Bits	Approximate Range
double	64	4.9e–324 to 1.8e+308
float	32	1.4e–045 to 3.4e+038

Example

```
// Compute the area of a circle.
class Area
{
public static void main(String args[])
{
double pi, r, a;
r = 10.8; // radius of circle
pi = 3.1416; // pi, approximately
a = pi * r * r; // compute area
System.out.println("Area of circle is " + a);
}
}
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac Area.java

C:\Program Files\Java\jdk1.8.0_40\bin>java Area
Area of circle is 366.436224
```

Characters

- Java uses Unicode to represent characters.
- Unicode defines a fully international character set that can represent all of the characters found in all human languages.
- Java char is a 16-bit type.
- The range of a char is 0 to 65,536.

Characters

```
// Demonstrate char data type.
class CharDemo
{
public static void main(String args[])
 {
char ch1, ch2;
ch1 = 88; // code for X
ch2 = 'Y';
System.out.print("ch1 and ch2: ");
System.out.println(ch1 + " " + ch2);
}
}
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac CharDemo.java

C:\Program Files\Java\jdk1.8.0_40\bin>java CharDemo
ch1 and ch2: X Y
```

Example

- Although char is designed to hold Unicode characters, it can also be thought of as an integer type on which you can perform arithmetic operations.

```
// char variables behave like integers.
class CharDemo2
{
public static void main(String args[])
 {
char ch1;
ch1 = 'X';
System.out.println("ch1 contains " + ch1);
ch1++; // increment ch1
System.out.println("ch1 is now " + ch1);
}
}
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac CharDemo2.java

C:\Program Files\Java\jdk1.8.0_40\bin>java CharDemo2
ch1 contains X
ch1 is now Y
```

Booleans

- boolean, used for logical values. It can have only one of two possible values, true or false.
- This is the type returned by all relational operators, as in the case of a < b.
- boolean is also the type required by the conditional expressions that govern the control statements such as if and for.

Code

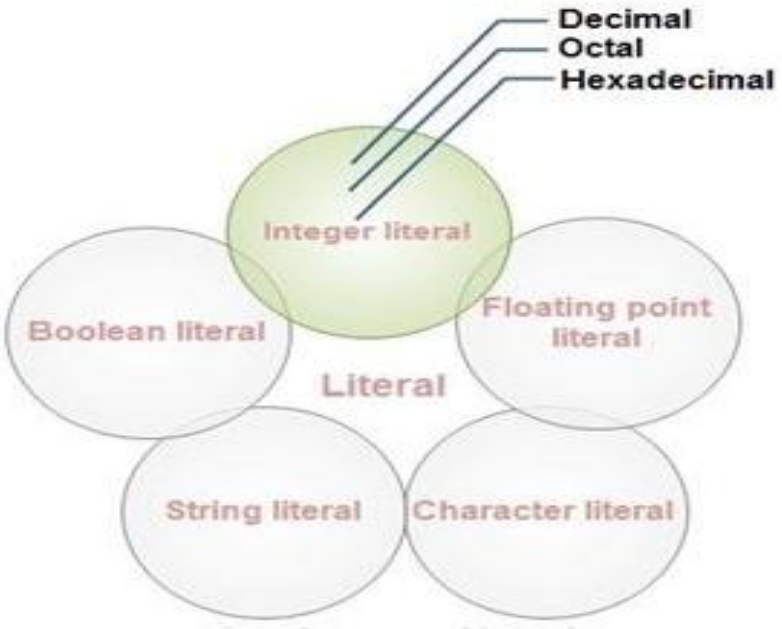
```
// Demonstrate boolean values.
class BoolTest
{
public static void main(String args[])
 {
boolean b;
b = false;
System.out.println("b is " + b);
b = true;
System.out.println("b is " + b);
if(b)
System.out.println("This is executed.");
System.out.println("10 > 9 is " + (10 > 9));
}
}
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac BoolTest.java

C:\Program Files\Java\jdk1.8.0_40\bin>java BoolTest
b is false
b is true
This is executed.
10 > 9 is true
```

Literals

- A constant value in Java is created by using a literal representation of it.
- A literal is a source code representation of a fixed value.
- Literals can be assigned to any primitive type variable.



Integer Literals

- Integer Literals are numbers that has no fractional pars or exponent.

We can specify integer constants in

- Decimal: any combination of digits taken from the set 0 to 9.

```
int a = 100;   //Decimal Constant
int b = -145   // A negative decimal constant
```

- Octal: any combination of digits taken from the set 0 to 7

```
int a = 0374;  //Octal Constant
int b = 097;  // Error: 9 is not an octal digit.
```

- Hexadecimal: A Sequence of digits begin the specification with 0x or 0X, shadowed by a system of digits in the range 0 to 9 and A (a) to F(f).

```
int  a = 0x34;
int b = -0XABF;
```

Floating-point Literals

- Floating-point Literals are also called as real constants.
- The Floating Point contains decimal points and can contain exponents.
- In the fractional form, the fractional number contains the integer part and fractional part. Example: `float x = 2.71`
- In the exponential form, the fractional number contains constants a mantissa and exponent. Mantissa contains the value of the number and the exponent contains the magnitude of the number. The exponent, if any present, specifies the magnitude of the number as a power of 10.

6.667	-11	6.667e-11	0.00000000006667

Character Literals

Escape Sequence	Description
\ddd	Octal character (ddd)
\uxxxx	Hexadecimal Unicode character (xxxx)
\'	Single quote
\"	Double quote
\\	Backslash
\r	Carriage return
\n	New line (also known as line feed)
\f	Form feed
\t	Tab
\b	Backspace

```
char ch = '\u0061';// Here /u0061 represent a.
```

String & Boolean Literals

- String Literals are treated as an array of char.
- By default, the compiler adds a special character called the 'null character' ('\0') at the end of the string to mark the end of the string.

```
String str = "good morning";
```

- A Boolean literal is either TRUE or FALSE.
- The value of 1 is assumed as true and the value of 0 is assumed as false.

```
boolean chosen = true;
boolean result = false;
```

VARIABLES

- In Java, all variables must be declared before they can be used.
- The basic form of a variable declaration:

  type identifier [= value][, identifier [= value] ...] ;

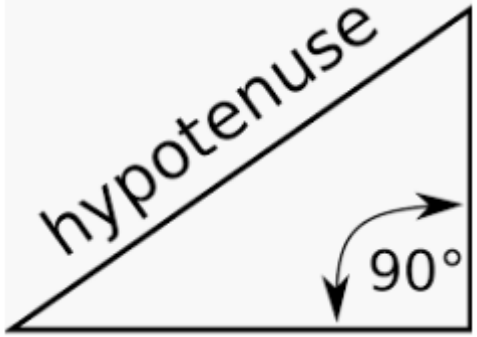
- The type is one of Java's atomic types, or the name of a class or interface.
- To declare more than one variable of the specified type, use a comma separated list.

Examples:
int a, b, c; // declares three ints, a, b, and c.
int d = 3, e, f = 5; // declares three more ints, initializing // d and f.
byte z = 22; // initializes z.
double pi = 3.14159; // declares an approximation of pi.
char x = 'x'; // the variable x has the value 'x'.

Dynamic Initialization

- Java allows variables to be initialized dynamically.
- Example:

```
class DynInit
{
public static void main(String args[])
{
double a = 3.0, b = 4.0;
// c is dynamically initialized
double c = Math.sqrt(a * a + b * b);
System.out.println("Hypotenuse is " + c);
}
}
```



a hypotenuse is the longest side of a right-angled triangle, the side opposite the right angle

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac DynInit.java

C:\Program Files\Java\jdk1.8.0_40\bin>java DynInit
Hypotenuse is 5.0
```

The Scope and Lifetime of Variables

- A block is begun with an opening curly brace and ended by a closing curly brace. A block defines a scope.
- A scope determines what objects are visible to other parts of your program. It also determines the lifetime of those objects.

There are two general categories of scopes:

- global and local
- variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope.
- Scopes can be nested. The outer scope encloses the inner scope

The Scope and Lifetime of Variables

```
// Demonstrate lifetime of a variable.
class LifeTime
{
public static void main(String args[])
{
int x;
for(x = 0; x < 3; x++)
 {
int y = -1; // y is initialized each time block is entered
System.out.println("y is: " + y); // this always prints -1
y = 100;
System.out.println("y is now: " + y);
}
}
}
```

Output:

```
y is: -1
y is now: 100
y is: -1
y is now: 100
y is: -1
y is now: 100
```

Type Conversion and Casting

- It is common to assign a value of one type to a variable of another type.
- If the two types are compatible, then Java will perform the conversion automatically.
- Possible to assign an int value to a long variable.
- No automatic conversion defined from double to byte.

Java's Automatic Conversions

An automatic type conversion will take place if the following two conditions are met:

- The two types are compatible.
- The destination type is larger than the source type.

Casting Incompatible Types

- General form:

(target-type) value

- if the size of the whole number component is too large to fit into the target integer type, then that value will be reduced modulo the target type's range.

```
int a;
byte b;
// ...
b = (byte) a;
```

Truncation

- when a floating-point value is assigned to an integer type, the fractional component is lost.

Demonstrate casts

```
class Conversion
{

public static void main(String args[])
 {
byte b;
int i = 257;
double d = 323.142;
System.out.println("\nConversion of int to byte.");
b = (byte) i;
System.out.println("i and b " + i + " " + b);
System.out.println("\nConversion of double to int.");
i = (int) d;
System.out.println("d and i " + d + " " + i);
System.out.println("\nConversion of double to byte.");
b = (byte) d;
System.out.println("d and b " + d + " " + b);
}
}
```

Output:

```
Conversion of int to byte.
i and b 257 1
Conversion of double to int.
d and i 323.142 323
Conversion of double to byte.
d and b 323.142 67
```

Summary

- Comments
- Data types
- Literals
- Variables
- Type casting

OPERATORS IN JAVA



ARITHMETIC OPERATORS

- Arithmetic operators are used in mathematical expressions.

Operator	Result
+	Addition
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
–=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement

ARITHMETIC OPERATORS

The Modulus Operator

- The modulus operator, %, returns the remainder of a division operation.

Arithmetic Compound Assignment Operators

- Java provides special operators that can be used to combine an arithmetic operation with an assignment.
- Thus, any statement of the form

var = var op expression;

can be rewritten as

var op= expression;

Increment and Decrement:

- The increment operator increases its operand by one.
- The decrement operator decreases its operand by one.
- These operators can appear in
  - postfix form, where operator follow the operand,
  - prefix form, where operator precede the operand.

EXAMPLE

```
public class Test
 {
   public static void main(String[] args)
 {
      int x=10,y=5;
      int a=x+y;
      int b=x-y;
      int c=x*y;
      int d=x/y;
      int e=x%y;
  System.out.println("sum="+a);
  System.out.println("Difference="+b);
  System.out.println("Product="+c);
  System.out.println("Quotient="+d);
  System.out.println("Remainder="+e);
        x+=y;
   System.out.println("x="+x);
     x-=y;
   System.out.println("x="+x);
```

```
a=1; b=2;
    c = ++b;
    d = a++;
    c++;
System.out.println("a = " + a);
System.out.println("b = " + b);
System.out.println("c = " + c);
System.out.println("d = " + d);
    }
 }
Output
sum=15
Difference=5
Product=50
Quotient=2
Remainder=0
x=15
x=10
a = 2
b = 3
c = 4
d = 1
```

THE BITWISE OPERATORS

- Java defines several bitwise operators that can be applied to the integer types, long, int, short, char, and byte.

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
|=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

Bitwise logical operators

- The bitwise logical operators are &, |, ^, and ~.
- The bitwise operators are applied to each individual bit within each operand.

A	B	A | B	A & B	A ^ B	~A
0	0	0	0	0	1
1	0	1	0	1	0
0	1	1	0	1	1
1	1	1	1	0	0

Bitwise left shift operator:

Left shift operator shift its value toward left as much number of bits as you defined and place zeros at the right most places.

Bitwise right shift operator:

In right shift, the Bit value for given number is simply moved to right side and left most bits are replaced with the zeros.

Bitwise Operator Compound Assignments:

- All of the binary bitwise operators have a compound form similar to that of the algebraic operators, which combines the assignment with the bitwise operation. For example, the following two statements, which shift the value in a right by four bits, are equivalent:

```
a = a >> 4;
a >>= 4;
```

Binary Coded Decimal

BCD is a weighted code and the weights used in binary coded decimal code are 8, 4, 2, 1, commonly called the **8421 code**

Binary Power	2^3	2^2	2^1	2^0
Binary Weight:	8	4	2	1

Decimal Number	BCD 8421 Code
0	0000 0000
1	0000 0001
2	0000 0010
3	0000 0011
4	0000 0100
5	0000 0101
6	0000 0110
7	0000 0111
8	0000 1000
9	0000 1001

EXAMPLE

```
public class Test {
   public static void main(String[] args)
{
     int a = 60; /* 60 = 0011 1100 */
     int b = 13; /* 13 = 0000 1101 */
     int c = 0;

     c = a & b;       /* 12 = 0000 1100 */
     System.out.println("a & b = " + c );

     c = a | b;       /* 61 = 0011 1101 */
     System.out.println("a | b = " + c );
 }
     c = a ^ b;       /* 49 = 0011 0001 */
     System.out.println("a ^ b = " + c );
```

```
     c = ~a;          /*-61 = 1100 0011 */
     System.out.println("~a = " + c );
     c = a << 2;      /* 240 = 1111 0000 */
     System.out.println("a << 2 = " + c );
c = a >> 2;      /* 15 = 1111 */
     System.out.println("a >> 2  = " + c );

     c = a >>> 2;     /* 15 = 0000 1111 */
     System.out.println("a >>> 2 = " + c );
   }
   Output:
a & b = 12
a | b = 61
a ^ b = 49
~a = -61
a << 2 = 240
a >> 2  = 15
a >>> 2 = 15
```

RELATIONAL OPERATORS

- The relational operators determine the relationship that one operand has to the other.
- The outcome of these operations is a boolean value.

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

EXAMPLE

```
public class Test
{    public static void main(String[] args)
{
    int a = 10;
     int b = 20;
     System.out.println("a == b = " + (a == b) );
     System.out.println("a != b = " + (a != b) );
     System.out.println("a > b = " + (a > b) );
     System.out.println("a < b = " + (a < b) );
     System.out.println("b >= a = " + (b >= a) );
     System.out.println("b <= a = " + (b <= a) );
   }
   }
```

Output:

a == b = false
a != b = true
a > b = false
a < b = true
b >= a = true
b <= a = false

BOOLEAN LOGICAL OPERATORS

- The Boolean logical operators operate only on boolean operands.
- All of the binary logical operators combine two boolean values to form a resultant boolean value.

Operator	Result
&	Logical AND
|	Logical OR
^	Logical XOR (exclusive OR)
||	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
|=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

BOOLEAN LOGICAL OPERATORS

A	B	A | B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

EXAMPLE

```
public class Test
{    public static void main(String[] args)
 {
      boolean a = true;
       boolean b = false;
       boolean c = a | b;
       boolean d = a & b;
       boolean e = a ^ b;
       boolean f = (!a & b) | (a & !b);
       boolean g = !a;
       System.out.println(" a = " + a);
       System.out.println(" b = " + b);
       System.out.println("  a|b = " + c);
       System.out.println("  a&b = " + d);
       System.out.println(" a^b = " + e);
      System.out.println("(!a&b)|(a&!b) = " + f);
       System.out.println("!a = " + g);
   }
}
```

Output:

```
 a = true
 b = false
 a|b = true
 a&b = false
 a^b = true
(!a&b)|(a&!b) = true
 !a = false
```

SHORT-CIRCUIT LOGICAL OPERATORS

- Java provides two secondary versions of the Boolean AND and OR operators, and are known as short-circuit logical operators.

- When short-circuit AND (&&) is used, if the first value is false, second value is not evaluated.

- Similarly for short-circuit OR (||), if the first value is true, then second value is not evaluated. These short-circuit operators will be useful when we want to control the evaluation of right hand operand.

EXAMPLE

```
public class Test
{
    public static void main(String[] args)
{
      int c = 0, d = 100, e = 50;
      if( c == 1 && e++ < 100 )
      {
         d = 150;
      }
      System.out.println("e = " + e );
  System.out.println("d = " + d );
   }

}
```

output:

e = 50

d=100

THE ASSIGNMENT OPERATOR

- The assignment operator is the single equal sign, =.

General form:

var = expression;

- Here, the type of var must be compatible with the type of expression.

THE ?: OPERATOR

- special ternary (three-way) operator

General form:

expression1 ? expression2 : expression3

- Here, expression 1 can be any expression that evaluates to a boolean value.
- If expression1 is true, then expression2 is evaluated; otherwise, expression3 is evaluated.
- The result of the ? operation is that of the expression evaluated.

EXAMPLE

```
public class Test
{
    public static void main(String[] args)
{
       int a=5,b=10;
          int c=(a>b)?1:0;
       System.out.println("c = " + c);
   }
   }
```

output:

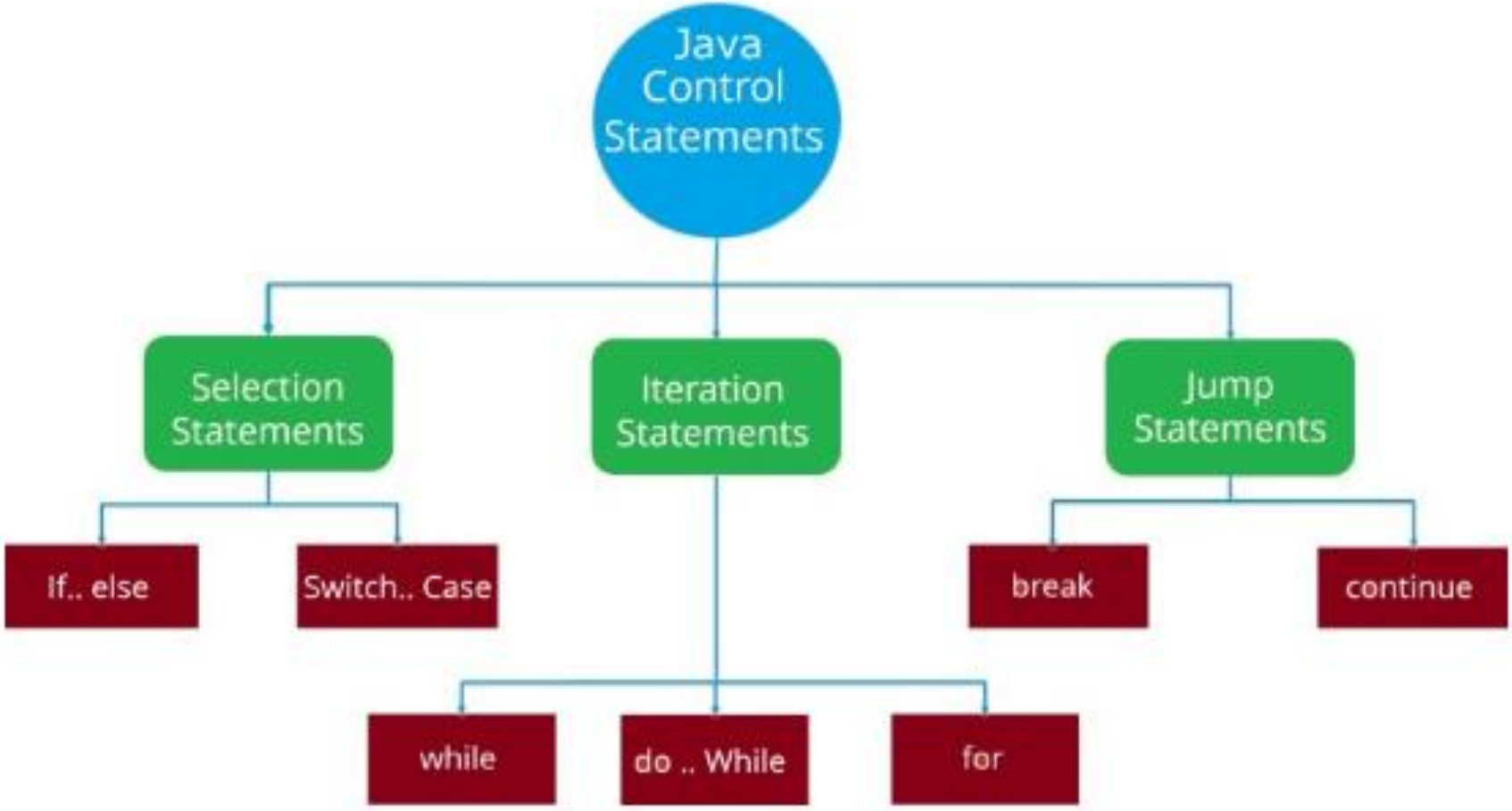
c=0

PRECEDENCE

Highest			
( )	[ ]	.	
++	– –	~	!
*	/	%	
+	–		
>>	>>>	<<	
>	>=	<	<=
==	!=		
&			
^			
|			
&&			
||			
?:			
=	op=		
Lowest			

CONTROL STATEMENTS IN JAVA

Types of statements



1. Selection statements

- Selection statements allow your program to choose different paths of execution based upon the outcome of an expression
- Java supports two selection statements:
  - if
  - switch

if statement

- The if statement is **Java's conditional branch statement.**
- It can be used to route program execution through two different paths.

General Form:

```
if (condition)
statement1;
else
statement2;
```

- Here, each statement may be a single statement or a compound statement enclosed in curly braces

Example

```
public class Test {
   public static void main(String[] args) {
      int n=5;
      if(n%2==0)
         System.out.println(n+" is even");
      else
         System.out.println(n+" is odd");
   }
 }
```

Output:

5 is odd

Nested if's

- A nested if is an if statement that is enclosed within another if statement.
- Example:

```
if(a<100)
{
    if(a<20)
    {
        a=1;
    }
}
```

Nested if's

- An "if" statement within another "if" statement is called a nested "if".

Syntax:

```
if(condition)
{
        statement(s);
        if(condition)
        {
                statement(s);
        }
        else
        {
                statement(s);
        }
}
else
{
        statement(s);
}
```

Outer if

Inner if

Inner else

Outer else

The if-else-if Ladder

General Form:

```
if(condition)
statement;
else if(condition)
statement;
else if(condition)
statement;
...
else
statement;
```

- The if statements are executed from the top down.
- The statement associated with true condition is executed, and the rest of the ladder is bypassed.
- The final else acts as a default condition;

Example

```
public class Test {
     public static void main(String[] args) {
           int n=5;
      if(n==0)
         System.out.println(n+" is zero");
      else if(n>0)
         System.out.println(n+" is positive");
      else
         System.out.println(n+" is negative");

   }

}
```

Output:

5 is positive

Switch statement

- The switch statement is Java's multi-way branch statement.
- It provides a better alternative than a large series of if-else-if statements.

General form:

```
switch (expression) {
case value1:
// statement sequence
break;
case value2:
// statement sequence
break;
...
case valueN:
// statement sequence
break;
default:
// default statement sequence
}
```

- The expression must be of type byte, short, int, or char;
- Each case value must be a unique literal (that is, it must be a constant, not a variable).
- Duplicate case values are not allowed.

Switch statement(contd…)

- The switch statement works like this:
  - The value of the expression is compared with each of the literal values in the case statements.
  - If a match is found, the code sequence following that case statement is executed.
  - If no match is found, then the default statement is executed. However, the default statement is optional.
  - The break statement is used inside the switch to terminate a statement sequence.

example

```
public class Test{
     public static void main(String[] args) {
        int a=10,b=5;
      char ch='/';
      switch(ch)
      {
        case '+':
           System.out.println("sum
is"+(a+b));
            break;
         case '-':
            System.out.println("Difference
is"+(a-b));
            break;
```

```
 case '*':
            System.out.println("Product
is"+(a*b));
            break;
          case '/':
            System.out.println("Quotient
is"+(a/b));
            break;
         }
     }
  }
```

2. **Iteration Statements**

- Java's iteration statements are
  - for
  - while
  - do-while
- These statements create loops.
- A **loop** repeatedly executes the same set of instructions until a termination condition is met.

while statement

- The while loop is Java's most fundamental loop statement.
- It repeats a statement or block while its controlling expression is true.

General form:

```
while(condition) {
// body of loop
}
```

- The condition can be any Boolean expression. The body of the loop will be executed as long as the conditional expression is true.

Example

```
public class Test {
    public static void main(String[] args) {
        int i=1;
    while(i<=10)
    {
       System.out.println(i);
       i++;
    }
   }
}
```

Output:

1
2
3
4
5
6
7
8
9
10

do-while statement

- If the conditional expression controlling a while loop is initially false, then the body of the loop will not be executed at all.
- The do-while loop always executes its body at least once, even if the conditional expression is false because its conditional expression is at the bottom of the loop.

General Form:

```
do {
// body of loop
} while (condition);
```

Example

```
public class Test {
    public static void main(String[] args) {
      int i=1;
  do
    {
       System.out.println(i);
       i++;
    }  while(i<=10);
  }
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

- **The do-while loop is especially useful when you process a menu selection, because you will usually want the body of a menu loop to execute at least once.**

Program:

```
import java.util.*;
public class Test {

   public static void main(String[] args) {
      int a=10,b=5;
      int ch;
      int choice;
      Scanner s=new Scanner(System.in);
      do
      {
         System.out.println("1.Add");
         System.out.println("2.sub");
         System.out.println("3.multiply");
         System.out.println("4.divide");
         System.out.println("Enter the value of ch");
         ch=s.nextInt();
```

```
switch(ch)
    {
       case 1:
          System.out.println("sum is"+(a+b));
          break;
       case 2:
          System.out.println("Difference is"+(a-b));
          break;
       case 3:
          System.out.println("Product is"+(a*b));
          break;
       case 4:
          System.out.println("Quotient is"+(a/b));
          break;
        }
    System.out.println("enter choice(0-exit, 1-continue)");
    choice=s.nextInt();
  }while(choice==1);
  }
}
```

For loop

General form of the traditional for statement:

for(initialization; condition; iteration) {

// body

}

- When the loop first starts, the initialization portion of the loop is executed.
- Next, condition is evaluated. This must be a Boolean expression.
- It usually tests the loop control variable against a target value.
- If this expression is true, then the body of the loop is executed. If it is false, the loop terminates.
- Next, the iteration portion of the loop is executed.
- This is usually an expression that increments or decrements the loop control variable.
- The loop then iterates, first evaluating the conditional expression, then executing the body of the loop, and then executing the iteration expression with each pass.
- This process repeats until the controlling expression is false.

example

```
public class Test {
   public static void main(String[] args)
{
          for(int i=1;i<=5;i++)
        System.out.println(i);
   }
}
```

Output:

1
2
3
4
5

For-each loop

General Form:

for(*type itr-var : collection*)

statement-block

- Here, *type* specifies the type
- *itr-var* specifies the name of an *iteration variable* that will receive the elements from a collection, one at a time, from beginning to end.
- The collection being cycled through is specified by *collection*

Example

```
public class Test {
     public static void main(String[] args)
{       int a[]={1,2,3,4,5};
       for(int x: a)
           System.out.println(x);
   }
}
```

Output:

1
2
3
4
5

Iterating Over Multidimensional Arrays

```
public class Test4 {
     public static void main(String[] args)
{
      int a[][]={{1,2,3},{4,5,6}};
           for(int x[]: a)
       {
          for(int y:x)
          {
          System.out.print(y+"\t");
          }
          System.out.println();
       }
   }
}
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac Test4.java

C:\Program Files\Java\jdk1.8.0_40\bin>java Test4
1       2       3
4       5       6
```

3.Jump Statements

- Java supports three jump statements:
  - **break**
  - **continue**
  - **return**
- These statements transfer control to another part of your program.

break statement

- In Java, the **break** statement has three uses.
  - First, it terminates a statement sequence in a **switch** statement.
  - Second, it can be used to exit a loop.
  - Third, it can be used as a “civilized” form of goto.

Using break to Exit a Loop

- When a break statement is encountered inside a loop, the loop is terminated and program control resumes at the next statement following the loop.

Example:

```
public class Test {
    public static void main(String[] args) {
     for(int i=1;i<=5;i++)
     {
        if(i==3)
           break;
        System.out.println("i="+i);
     }
     System.out.println("LOOP complete");
     }
}
```

Output:

```
i=1
i=2
LOOP complete
```

Using break as a Form of Goto

- Java does not have a goto statement because it provides a way to branch in an arbitrary and unstructured manner.
- Java defines an expanded form of the **break** statement.
- By using this form of **break**, you can, for example, break out of one or more blocks of code.
- These blocks need not be part of a loop or a **switch**. They can be any block.

Using break as a Form of Goto

- The general form of the labeled **break** statement

  break label;

- Most often, label is the name of a label that identifies a block of code.
- When this form of break executes, control is transferred out of the named block.
- The labeled block must enclose the break statement, but it does not need to be the immediately enclosing block.
- A label is any valid Java identifier followed by a colon.
- Once you have labeled a block, you can then use this label as the target of a break statement.

Example

```
public class Test {
     public static void main(String[] args) {
      boolean t = true;
   first: {
   second: {
   third: {
      System.out.println("Before the break.");
      if(t) break first;
       System.out.println("This won't execute");
          }
      System.out.println("This won't execute");
   }
      System.out.println("This is after second block.");
   }
   }
}
```

Example

```
public class Test
{
    public static void main(String[] args)
 {
     boolean t = true;
   first: {
   second: {
   third: {
      System.out.println("Before the break.");
      if(t) break first;
       System.out.println("This won't execute");
          }
      System.out.println("This won't execute");
   }
      System.out.println("This is after second block.");
   }
  }
}
```

If break label is first, output is:
Before the break.
If break label is second, output is:
Before the break.
This is after second block.
If break label is third, output is:
Before the break.
This won't execute
This is after second block.

continue statement

- If you want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration, use continue statement.
- Example:

```
public class Test {
    public static void main(String[] args) {
            for(int i=1; i<10; i++) {
            if (i%2 == 0)
            {
                continue;
            }
            else
              System.out.print( i +" ");
            }
   }
}
```

continue statement

- If you want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration, use continue statement.
- Example:

```
public class Test {
    public static void main(String[] args) {
        for(int i=1; i<10; i++) {
        if (i%2 == 0)
        {
            continue;
        }
        else
          System.out.print( i +" ");
        }
   }
}
```

Output:
1 3 5 7 9

return statement

- The return statement is used to explicitly return from a method.
- That is, it causes program control to transfer back to the caller of the method.
- The return statement immediately terminates the method in which it is executed.

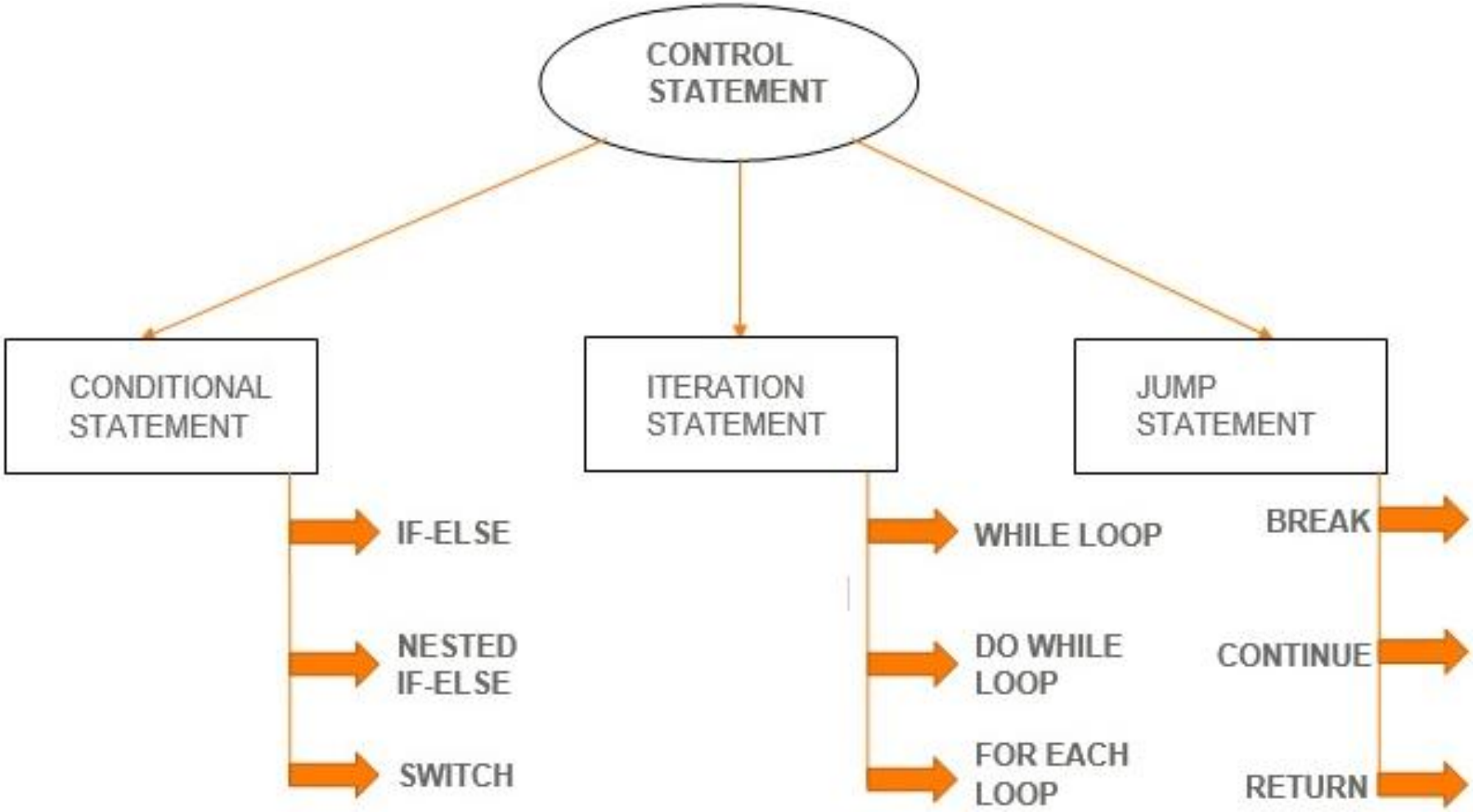
EXAMPLE:

```
public class JavaApplication1 {
    public static void main(String[] args) {
     boolean t = true;
     System.out.println("Before the return.");
if(t) return; // return to caller
System.out.println("This won't execute.");
   }
}
```

Output:

Before the return.

Summary


CONTROL STATEMENT
CONDITIONAL STATEMENT
ITERATION STATEMENT
JUMP STATEMENT
IF-ELSE
NESTED IF-ELSE
SWITCH
WHILE LOOP
DO WHILE LOOP
FOR EACH LOOP
BREAK
CONTINUE
RETURN

Classes in Java

INTRODUCING CLASSES

The General Form of a Class:

```
class classname {
    type instance-variable-1;
    ...
    type instance-variable-n;

    type method-name-1(parameter-list)        { ... }
    type method-name-2(parameter-list)        { ... }
    ...
    type method-name-m(parameter-list)        { ... }
}
```

Declaring Objects

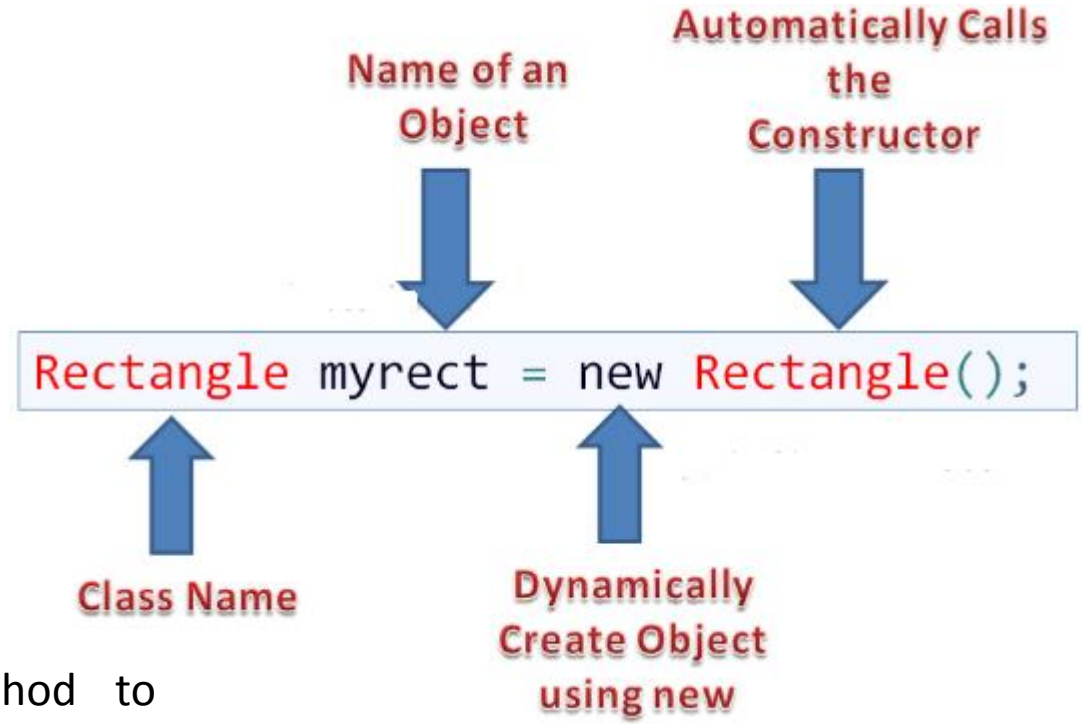
- When you create a class, you are creating a new data type. You can use this type to declare objects of that type.

Steps for creating object

- First, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can refer to an object.
- Second, you must acquire an actual physical copy of the object and assign it to that variable. You can do this using the new operator.
- The **new operator** dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it.

Declaring Objects

```
class Rectangle
{
double length;
double breadth;
}
Rectangle myrect = new Rectangle();
```



- A constructor is a special method to initialize an object.
- Classes can explicitly define their own constructors within their class definition.
- However, if no explicit constructor is specified, then Java will automatically supply a default constructor.

```
//Accessing instance variables and methods
public class Sum {
  int a,b,c;
  void operation()
  {
     c=a+b;
  }
  void print()
  {
     System.out.println("Sum of  "+a+" and  "+b+" is  "+c);
  }
}
public class Test {
     public static void main(String[] args) {
      Sum s1=new Sum();
      s1.a=10;
      s1.b=10;
      s1.operation();
      s1.print();
  }
}
```

Output:
Sum of 10 and 10 is 20

```
import java.util.*;
public class Sum {
   int a,b,c;
   void read()
   {
       Scanner s=new Scanner(System.in);
       System.out.println("Enter x");
       a=s.nextInt();
        System.out.println("Enter y");
       b=s.nextInt();
   }
  void operation()
   {
       c=a+b;
   }
  void print()
  {
      System.out.println("Sum of  "+a+" and
"+b+" is  "+c);
  }
}
```

```
public class JavaApplication1 {

    public static void main(String[] args) {
       Sum s1=new Sum();
       s1.read();
       s1.operation();
       s1.print();
    }
   }
```

Output:

```
Enter x
5
Enter y
2
Sum of  5 and  2 is  7
```

Assigning Object Reference Variables

- When you assign one object reference variable to another object reference variable, you are not creating a copy of the object; you are only making a copy of the reference.

Example:

```
Sum b1=new Sum(x,y);
Sum b2=b1;
```

- b1 and b2 will both refer to the same object.
- It did not allocate any memory or copy any part of the original object.
- It simply makes b2 refer to the same object as does b1.
- Any changes made to the object through b2 will affect the object to which b1 is referring, since they are the same object.

JAVA ACCESS SPECIFIERS

- The access to classes, constructors, methods and fields are regulated using access modifiers i.e. a class can control what information or data can be accessible by other classes.
- A member has package or default accessibility when no accessibility modifier is specified.

Access Modifiers

1. private
2. protected
3. default
4. public

public access modifier(Least restrictive)

- Fields, methods and constructors declared public within a public class are **visible to any class in the Java program**, whether these classes are in the same package or in another package.

private access modifier(most restrictive)

- The private fields or methods cannot be used for classes and Interfaces.
- Fields, methods or constructors declared private are strictly controlled, which means they cannot be accesses by anywhere outside the enclosing class.
- A standard design strategy is to make all fields private and provide public getter methods for them.

protected access modifier

- The protected keyword cannot be used for classes and Interfaces. It also cannot be used for fields and methods within an interface.
- Fields, methods and constructors declared protected in a superclass can be accessed only by subclasses in other packages.
- Classes in the same package can also access protected fields, methods and constructors as well, even if they are not a subclass of the protected member's class.

default access modifier

- Java provides a default specifier which is used when no access modifier is present.
- Any class, field, method or constructor that has no declared access modifier is accessible only by classes in the same package.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Implementing a Complex Number Class

```
public class Complex
{
   // State  (instance variables)
   // Behaviour(instance methods)
  // constructors
  // accessors
  // mutators
  // operations/messages
}
```

Implementing State

- An object's state is contained in instance variables

```
// State

   private double real;
   private double imag;
```



- Each object created from this class will have separate occurrences of the instance variables

Implementing Behaviour: Constructors

- Constructors are methods that describe the initial state of an object created of this class.

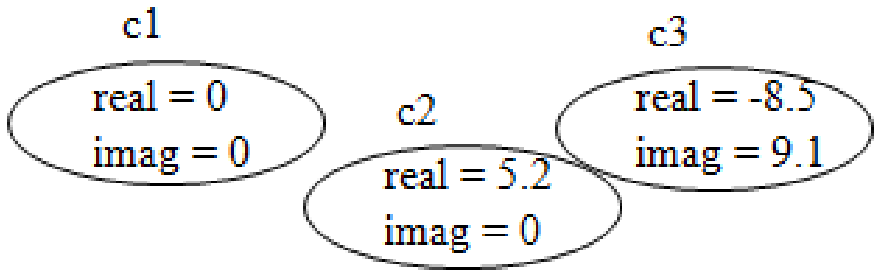
Default Constructor

```
public Complex()
{
   real = 0.0; imag = 0.0;
}
public Complex(double rp)
{
     real = rp; imag = 0.0;
}
public Complex(double rp, double ip)
{
     real = rp; imag = ip;
}
```

Implementing Behaviour: Constructors

- Constructors are invoked only when the object is created. They cannot be called otherwise.

```
Complex c1, c2, c3;
```




```
c1 = new Complex();
c2 = new Complex(5.2);
c3 = new Complex(-8.5, 9.1)
```

Constructors initialize the state of an object upon creation.

METHODS

General form of a method:

returntype name(parameter-list)

{

// body of method

}

TYPES OF METHODS

- Mutator methods

Methods that change instance fields are called mutator methods. A common convention is to prefix mutator methods with the prefix set.

- Accessor Methods

Methods that only access instance fields without modifying them are called accessor methods. A common convention is to prefix accessor methods with the prefix get.

```
public class Student {
   String name;
   int rollno;
   String getName()
   {       return name;
   }
   int getRollno()
   {       return rollno;
   }
   void setName(String s)
   {       name=s;
   }
   void setRollno(int a)
   {       rollno=a;
   }
}
public class Example {
   public static void main(String[] args) {
      Student s1=new Student();
      s1.setName("Danica");
      s1.setRollno(21);
      System.out.println("Name is: "+s1.getName());
      System.out.println("Rollno is: "+s1.getRollno());
   }
}
```

Output:
Name is: Danica
Rollno is: 21

METHOD OVERLOADING

- In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. The methods are said to be overloaded, and the process is referred to as method overloading.
- Method overloading is one of the ways that Java supports polymorphism.
- overloaded methods must differ in the type and/or number of parameters
- When Java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.

```
public class Test {
    int sum(int a,int b)
    {
       return(a+b);
    }
    double sum(double a,double b)
    {
        return(a+b);
    }
    String sum(String a,String b)
    {
        return(a+b);
    }
}
```

```
public class Example {
    public static void main(String[]
args) {
    Test t1=new Test();
     System.out.println("Sum of two
int  "+t1.sum(2,3));
     System.out.println("Sum of two
double "+t1.sum(2.5,3.8));
     System.out.println("Sum of two
string "+t1.sum("wel","come"));
    }
}
```

USING OBJECTS AS PARAMETERS

EXAMPLE:

```
public class complex {
   int x,y;
   public complex()
   {
     x=0;
     y=0;
   }
   public complex(int a,int b)
   {
      x=a;
      y=b;
   }
   complex add(complex c1,complex c2)
   {
      complex t=new complex();
      t.x=c1.x+c2.x;
      t.y=c1.y+c2.y;
      return t;
   }
   void print()
   {
      System.out.println("a is:  "+x);
      System.out.println("b is:  "+y);
    }
}
public class Example {
   public static void main(String[] args)
{
       complex c1=new complex(2,2);
       complex c2=new complex(3,3);
       complex c3=new complex();
       c3=c3.add(c1, c2);
       System.out.println("c1:");
       c1.print();
       System.out.println("c2:");
       c2.print();
       System.out.println("c3:");
       c3.print();
    }
```

ARGUMENT PASSING METHODS

- In general, there are two ways that a computer language can pass an argument to a subroutine.
- Call by value

This approach copies the value of an argument into the formal parameter of the subroutine. Therefore, changes made to the parameter of the subroutine have no effect on the argument.

- Call by Reference

In this approach, a reference to an argument (not the value of the argument) is passed to the parameter. Inside the subroutine, this reference is used to access the actual argument specified in the call. This means that changes made to the parameter will affect the argument used to call the subroutine.

- **Java uses both approaches, depending upon what is passed.**

```
public class Test {
   void add(int a,int b)
   {
      a=a+5;
      b=b+5;
   }
}
public class Example {
   public static void main(String[] args) {
      Test t1=new Test();
      int x=5,y=10;
       System.out.println("Before method call");
       System.out.println("x="+x);
       System.out.println("y="+y);
      t1.add(x,y);
      System.out.println("After method call");
      System.out.println("x="+x);
      System.out.println("y="+y);
        }
  }
```

Output:
Before method call
x=5
y=10
After method call
x=5
y=10

```
public class Test {
   int a,b;
   Test(int x,int y)
   {
      a=x;
      b=y;
   }
   void add(Test t)
   {
      t.a=t.a+5;
      t.b=t.b+5;
   }
}
public class Example {
   public static void
main(String[] args) {
      Test t1=new Test(2,3);
      System.out.println("Before
method call");

System.out.println("a="+t1.a);

System.out.println("b="+t1.b);
       t1.add(t1);
       System.out.println("After
method call");

System.out.println("a="+t1.a);

System.out.println("b="+t1.b);
    }
}
```

Output:
Before method call
a=2
b=3
After method call
a=7
b=8

ARRAYS

ARRAYS

- An array is a group of similar variables that are referred to by a common name.
- Arrays of any type can be created and may have one or more dimensions.
- A specific element in an array is accessed by its index.
- 1 D array is a list of similar variables.

Creation of 1 D array:

Obtaining an array is a two-step process.

- **Declare** a variable of the desired array type.
- **Allocate the memory** that will hold the array, using new and assign it to the array variable.

General form

- type var-name[ ];

Example:

- int days[];

Allocation of memory:

- **new** is a special operator that allocates memory.

General form of new:

- array-var = new type[size];

The elements in the array allocated by new will automatically be initialized to zero.

Example:

days = new int[12];

type var-name[ ]= new type[size];

```
public class ArrayTest
{
   public static void main(String[] args)
 {
      int days[]=new int[5];
      days[0]=1;
      days[1]=2;
      days[2]=3;
      days[3]=4;
      days[4]=5;
      System.out.println("days[4]="+days[4]);
   }
}
```

Output:

days[4]=5

Array Initialization:

- Arrays can be initialized when they are declared.
- An **array initializer** is a list of comma-separated expressions surrounded by curly braces. The commas separate the values of the array elements.
- The array will automatically be created large enough to hold the number of elements you specify in the array initializer.
- There is no need to use new.

```
public class ArrayTest
{
  public static void main(String[] args)
{
     int days[]={1,2,3,4,5};
     System.out.println("days[4]="+days[4]);
   }
 }
```

Example to perform the sum of elements in an array

```
public class ArrayTest
{
    public static void main(String[] args)
 {
     int days[]={1,2,3,4,5};
     int sum=0;
     for(int i=0;i<5;i++)
        sum+=days[i];
     System.out.println("sum="+sum);
   }
 }
```

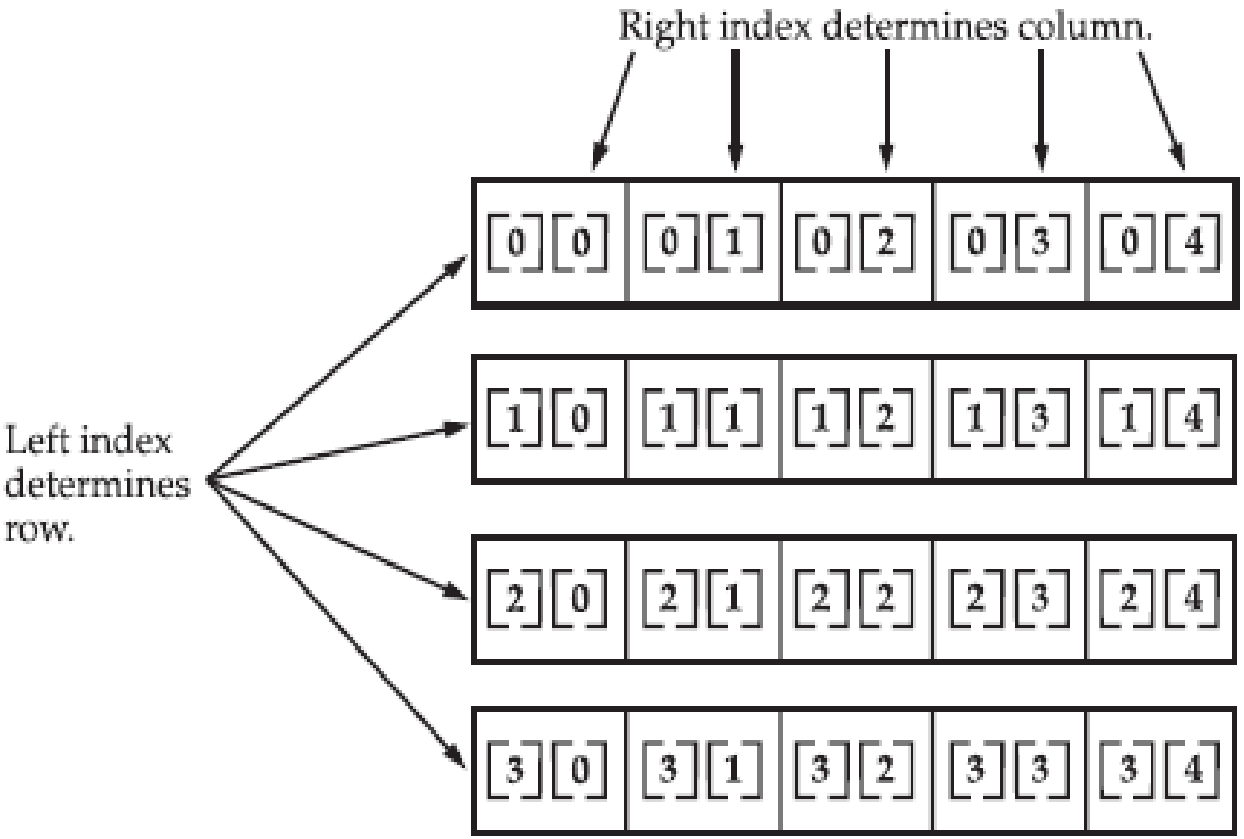
Multidimensional Arrays

- Java has no multidimensional arrays. Multidimensional arrays are faked as “arrays of arrays.”
- To declare a multidimensional array variable, specify each additional index using another set of square brackets.
- For example, the following declares a twodimensional array variable called twoD.

```
int twoD[][] = new int[4][5];
```

- This allocates a 4 by 5 array and assigns it to twoD. Internally this matrix is implemented as an array of arrays of integer type.

Multidimensional Arrays



Given: int twoD [ ] [ ] = new int [4] [5] ;

```
public class MultiArrayTest
{
    public static void main(String[] args)
 {          int a[][]=new int[2][2];
           for(int i=0;i<2;i++)
     {
        for(int j=0;j<2;j++)
        {              a[i][j]=1;
        }
     }
     for(int i=0;i<2;i++)
     {
        for(int j=0;j<2;j++)
        {     System.out.print("\t"+a[i][j]);
        }
        System.out.println();
     }   } }
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>java MultiArrayTest
        1       1
        1       1
```

When you allocate memory for a multidimensional array, you need only specify the memory for the first (leftmost) dimension. You can allocate the remaining dimensions separately.

For example,

```
int twoD[][] = new int[4][];
twoD[0] = new int[5];
twoD[1] = new int[5];
twoD[2] = new int[5];
twoD[3] = new int[5];
```

Ragged Arrays

- Ragged Arrays are arrays in which different rows have different lengths.
- Example:

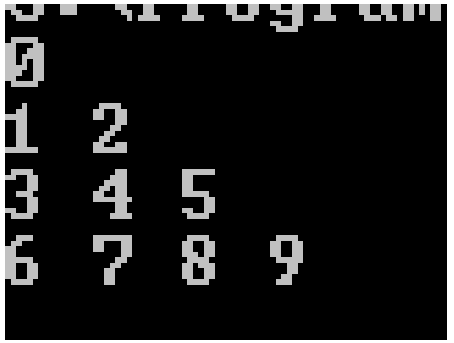
```
int a[][]=new int[4][];
a[0]=new int[1];//row 1 is of length 1
a[1]=new int[2];//row 2 is of length 2
a[2]=new int[3];//row 3 is of length 3
a[3]=new int[4];//row 4 is of length 4
```

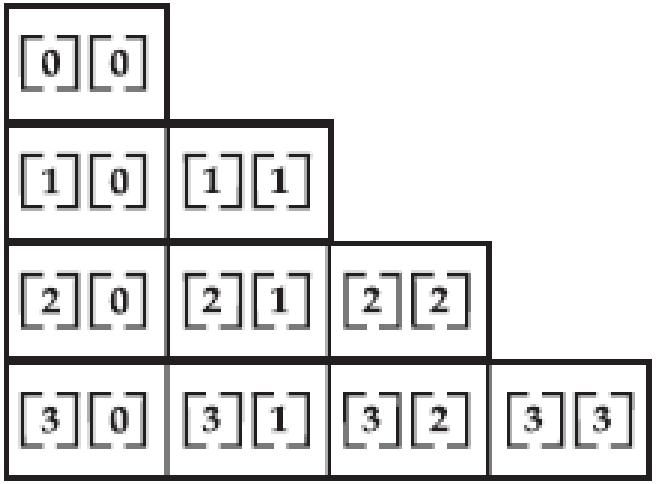
```
public class Multitest {
    public static void main(String[] args) {
      int a[][]=new int[4][];
      a[0]=new int[1];
      a[1]=new int[2];
      a[2]=new int[3];
      a[3]=new int[4];
            int i,j;
     for(i=0;i<4;i++)
     {
        for(j=0;j<i+1;j++)
        {
            a[i][j]=j+1;
        }
     }
    for(i=0;i<4;i++)
    {
        for(j=0;j<i+1;j++)
                        {
          System.out.print(a[i][j]+"\t");
                        }
      System.out.println();
    }   }  }
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>javac Multitest.java

C:\Program Files\Java\jdk1.8.0_40\bin>java Multitest
1
1       2
1       2       3
1       2       3       4
```

```
public class TwoDAgain {
    public static void main(String[] args) {
      int a[][]=new int[4][];
      a[0]=new int[1];
      a[1]=new int[2];
      a[2]=new int[3];
      a[3]=new int[4];
            int i,j,k=0;
     for(i=0;i<4;i++)
     {
        for(j=0;j<i+1;j++)
        {
            a[i][j]=k;
            k++;
        }
     }
    for(i=0;i<4;i++)
    {
        for(j=0;j<i+1;j++)
                        {
          System.out.print(a[i][j]+"  ");
                        }
      System.out.println();
    }   }   }
```





Alternative Array Declaration Syntax

- There is a second form that may be used to declare an array:

  type[ ] var-name;

- Here, the square brackets follow the type specifier, and not the name of the array variable.
- For example, the following two declarations are equivalent:

  int al[] = new int[3];

  int[] a2 = new int[3];

- The following declarations are also equivalent:

  char twod1[][] = new char[3][4];

  char[][] twod2 = new char[3][4];

- This alternative declaration form offers convenience when declaring several arrays at the same time.
- For example,

  int[] nums, nums2, nums3; // create three arrays

Anonymous array:

- It is possible to initialize an anonymous array:

  new int[] { 17, 19, 23, 29, 31, 37 }

- This expression allocates a new array and fills it with the values inside the braces. It counts the number of initial values and sets the array size accordingly. You can use this syntax to reinitialize an array without creating a new variable. For example,

  a = new int[] { 17, 19, 23, 29, 31, 37 };

Program:

```
class Test
{
public static void main(String[] args)
{
                int[] a=new int[]{1,2,3,4,5};
                for(int i=0;i<5;i++)
                System.out.println(a[i]);
}  }
```

Arrays class

The Arrays class of the java.util package contains several static methods that we can use to fill, sort, search, etc in arrays.

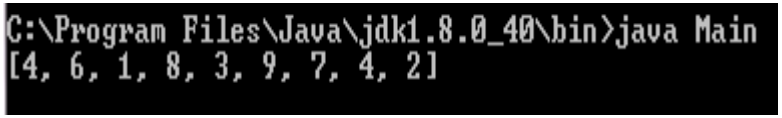
public static String toString(int[] a)

The string representation consists of a list of the array's elements, enclosed in square brackets ("[]"). Adjacent elements are separated by the characters, a comma followed by a space.

Arrays class

Example:

```
import java.util.Arrays;
 public class Main
{
    public static void main(String[] args)
    {
        int ar[] = {4, 6, 1, 8, 3, 9, 7, 4, 2};
        System.out.println(Arrays.toString(ar));
    }
}
```



Arrays class

- **public static void sort(int[] a)** – Sorts the specified array into ascending numerical order.
- **public static void sort(int[] a, int fromIndex, int toIndex)**
- If we wish to sort a specified range of the array into ascending order. we can use this. The range to be sorted extends from fromIndex, inclusive, to toIndex, exclusive. If fromIndex == toIndex, the range to be sorted is empty.

Arrays class

- **Example**

```
C:\Program Files\Java\jdk1.8.0_40\bin>java Main1
array in 0-4 range[1, 4, 6, 8, 3, 9, 7, 4, 2]
Completely sorted order[1, 2, 3, 4, 4, 6, 7, 8, 9]
```

```
import java.util.Arrays;
public class Main1
public static void main(String[] args)
    {
        int ar[] = {4, 6, 1, 8, 3, 9, 7, 4, 2};
        Arrays.sort(ar, 0, 4);
        System.out.println("array in 0-4 range" + Arrays.toString(ar));
        Arrays.sort(ar);
    System.out.println("Completely sorted order" + Arrays.toString(ar));
    }
```

Arrays class

public static int[] copyOf(int[] original, int newLength)

Copies the specified array and length. It truncates the array if provided length is smaller and pads if provided length is greater.

public static int[] copyOfRange(int[] original, int from, int to)

Copies the specified range of the specified array into a new array. The initial index of the range (from) must lie between zero and original.length, inclusive.

Arrays class

```
import java.util.Arrays;
public class Main2
public static void main(String[] args)
{
    int ar[] = {4, 6, 1, 8, 3, 9, 7, 4, 2};
    int[] copy = Arrays.copyOf(ar, ar.length);
 System.out.println("Copied array “+ Arrays.toString(copy));
    int[] rcopy = Arrays.copyOfRange(ar, 1, 5);
    System.out.println("subarray “+ Arrays.toString(rcopy));
}
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>java Main2
Copied array[4, 6, 1, 8, 3, 9, 7, 4, 2]
subarray[6, 1, 8, 3]
```

Arrays class

- **public static void fill(int[] a, int val)**

Fills all elements of the specified array with the specified value.

- **public static void fill(int[] a, int fromIndex, int toIndex, int val)**

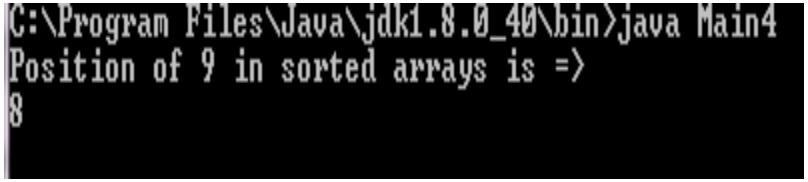
– Fills elements of the specified array with the specified value from the fromIndex element, but not including the toIndex element.

Arrays class

```
import java.util.Arrays;
 public class Main3
public static void main(String[] args)
   {
      int ar[] = {4, 6, 1, 8, 3, 9, 7, 4, 2};
      Arrays.fill(ar, 0, 3, 0);
 System.out.println("Array filled with 0 "+ Arrays.toString(ar));
      Arrays.fill(ar, 10);
      System.out.println("Array completely filled with 10”
+Arrays.toString(ar));
   }
```

```
C:\Program Files\Java\jdk1.8.0_40\bin>java Main3
Array filled with 0 [0, 0, 0, 8, 3, 9, 7, 4, 2]
Array completely filled with 10[10, 10, 10, 10, 10, 10, 10, 10, 10]
```

public static int binarySearch(int[] a, int key) Returns an int value for the index of the specified key in the specified array. Returns a negative number if the specified key is not found in the array. For this method to work properly, the array must first be sorted by the sort method.




```
import java.util.Arrays;
 public class Main4
public static void main(String[] args)
   {
      int ar[] = {4, 6, 1, 8, 3, 9, 7, 4, 2};
      Arrays.sort(ar);
      int index = Arrays.binarySearch(ar,9);
      System.out.println("Position of 9 in sorted"+ " arrays is => \n"
+ index);
    }
```

JAVA
PACKAGES
&
DOCUMENTATION

JAVA Packages

PACKAGES

- Group of classes in a collection - *package*.
- The package can then be further subdivided into subpackages.

Packages are convenient

- for organizing our work
- for separating our work from code libraries provided by others.

- The standard Java library is distributed over a number of packages, including java.lang, java.util, java.net, and so on.
- The standard Java packages are examples of hierarchical packages.
- All standard Java packages are inside the java and javax package hierarchies.

The main reason for using packages is

- To guarantee the uniqueness of class names.

Defining a package

To create a package …

- include a package command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package.
- The package statement defines a name space in which classes are stored. If you omit the package statement, the class names are put into the default package, which has no name.
- This is the general form of the package statement:

```
package pkg;
```

- Here, pkg is the name of the package.

- Java uses file system directories to store packages. More than one file can include the same package statement. The package statement simply specifies to which package the classes defined in a file belong.
- You can create a hierarchy of packages. To do so, simply separate each package name from the one above it by use of a period. The general form of a multileveled package statement is shown here:
  - package pkg1[.pkg2[.pkg3]];
- A package hierarchy must be reflected in the file system of your Java development system. For example, a package declared as
  - package java.awt.image;
- needs to be stored in java\awt\image in a Windows environment

- For example, the file Employee.java in

```
package p1;
public class Employee
{
        . . .
}
```

- If you don't put a package statement in the source file, then the classes in that source file belong to the default package. The default package has no package name.
- Place source files into a subdirectory that matches the full package name.
- For example, all source files in the package p1 package should be in a subdirectory p1. The compiler places the class files into the same directory structure.

```
package p1.p2;
public class Employee
{
        . . .
}
```

- All source files in the package p1 package should be in a subdirectory p1/p2 on Windows.

Class Importation

- A class can use all classes from its own package and all public classes from other packages.
- You can access the public classes in another package in two ways.

1. Simply to add the full package name in front of every class name

For example:

- java.util.Date today = new java.util.Date();

This is obviously tedious.

2. Use the import statement

- You can import a specific class or the whole package. You place import statements at the top of your source files (but below any package statements). For example, you can import all classes in the java.util package with the statement
- import java.util.*;

Then you can use

- Date today = new Date();

without a package prefix.

- You can also import a specific class inside a package:
- import java.util.Date;

```
package p1;
public class Sum
{
        int a,b;
        public Sum()
        {
                a=0;b=0;
        }
        public Sum(int x,int y)
        {
                a=x;
                b=y;
        }
        public void add()
        {
    System.out.println("sum="+(a+b));
        }
}
```

```
import p1.*;
class Test
{
public static void main(String[] args)
        {
                Sum a1=new Sum(3,5);
                a1.add();
        }
}
```

```
package p1.p2;
public class Sum
{
        int a,b;
        public Sum()
        {
                a=0;b=0;
        }
        public Sum(int x,int y)
        {
                a=x;
                b=y;
        }
        public void add()
        {
System.out.println("sum="+(a+b));
        }
}
```

```
import p1.p2.*;
class Test
{
public static void main(String[] args)
        {
        Sum a1=new Sum(3,5);
                a1.add();
        }
}
```

JAVA DOCUMENTATION COMMENTS

JAVA DOCUMENTATION COMMENTS

- The JDK contains a very useful tool, called javadoc.
- It generates HTML documentation from source files.
- If we add comments that start with the special delimiter /** to our source code, then we can easily produce professional looking documentation.
- When documentation comments are in the same file as the source code, it is an easy to update both and run javadoc again.

1. Comment Insertion

- The javadoc utility extracts information for the following items:
  - Packages
  - Public classes and interfaces
  - Public and protected fields
  - Public and protected constructors and methods
- We can write a comment for each of these features.
- Each comment is placed immediately above the feature it describes.
- A comment starts with a /** and ends with a */.

1. Comment Insertion contd..

- Each /** . . . */ documentation comment contains free-form text followed by tags.
- A tag starts with an @, such as @author or @param.
- The first sentence of the free-form text should be a summary statement.
- The javadoc utility automatically generates summary pages that extract these sentences.
- In the free-form text, we can use HTML modifiers such as ... for emphasis,<code>...</code> for a monospaced “typewriter” font, ... for strong emphasis, and even <img ...> to include an image.

2. Class Comments

- The class comment must be placed after any import statements, directly before the class definition.
- Example:

```
/**
* A <code>Card</code> object represents a playing card, such
* as "Queen of Hearts". A card has a suit (Diamond, Heart,
* Spade or Club) and a value (1 = Ace, 2 . . . 10, 11 = Jack,
* 12 = Queen, 13 = King)
*/
public class Card
{
. . .
}
```

3. Method Comments

- Each method comment must immediately precede the method that it describes. In addition to the general-purpose tags, you can use the following tags:

@param variable description

- This tag adds an entry to the “parameters” section of the current method. The description can span multiple lines and can use HTML tags. All @param tags for one method must be kept together.

@return description

- This tag adds a “returns” section to the current method. The description can span multiple lines and can use HTML tags.

@throws class description

- This tag adds a note that this method may throw an exception.

3. Method Comments

- Example:

```
/**
* Raises the salary of an employee.
* @param byPercent the percentage by which to raise the salary (e.g. 10
means 10%)
* @return the amount of the raise
*/
public double raiseSalary(double byPercent)
{
double raise = salary * byPercent / 100;
salary += raise;
return raise;
}
```

4. Field Comments

- Used to document public fields—generally static constants.

- For example:

```
/**
* The "Hearts" card suit
*/
public static final int HEARTS = 1;
```

5. General Comments

The following tags can be used in class documentation comments:

@author name

- This tag makes an “author” entry. You can have multiple @author tags, one for each author.

@version text

- This tag makes a “version” entry. The text can be any description of the current version.

5. General Comments

The following tags can be used in all documentation comments:

@since text

- This tag makes a “since” entry. The text can be any description of the version that introduced this feature.
- For example, @since version 1.7.

@deprecated text

- This tag adds a comment that the class, method, or variable should no longer be used. The text should suggest a replacement. For example:
- @deprecated Use <code>setVisible(true)</code> instead

5. General Comments

- You can use hyperlinks to other relevant parts of the javadoc documentation, or to external documents, with the @see and @link tags.

@see reference

- This tag adds a hyperlink in the “see also” section. It can be used with both classes and methods.
- Here, reference can be one of the following:

➢ package.class #feature label

➢ label

➢ "text"

5. General Comments

- We can supply the name of a class, method, or variable, and javadoc inserts a hyperlink to the documentation.
- For example,

@see com. Horstmann. corejava.Employee#raiseSalary(double)

- makes a link to the raiseSalary(double) method in the

com. Horstmann.corejava.Employee class.

- You can omit the name of the package, or both the package and class name. Then, the feature will be located in the current package or class.
- Note that you must use a #, not a period, to separate the class from the method or variable name.

5. General Comments

- If the @see tag is followed by a < character, then you need to specify a hyperlink. You can link to any URL you like. For example:
- @see The Core Java home page
- If the @see tag is followed by a " character, then the text is displayed in the “see also” section.
- For example:
- @see "Core Java 2 volume 2”

5. General Comments

- You can add multiple @see tags for one feature, but you must keep them all together.
- If you like, you can place hyperlinks to other classes or methods anywhere in any of your documentation comments. Insert a special tag of the form
- {@link package.class#feature label}
- anywhere in a comment.
- The feature description follows the same rules as for the @see tag.

6. Package and Overview Comments

- You place the class, method, and variable comments directly into the Java source files, delimited by /** . . . */ documentation comments.
- However, to generate package comments, you need to add a separate file in each package directory.

Two choices:

1. Supply an HTML file named package.html. All text between the tags <body>...</body> is extracted.
2. Supply a Java file named package-info.java.

   The file must contain an initial Javadoc comment, delimited with /** and */, followed by a package statement. It should contain no further code or comments.

6. Package and Overview Comments

- You can also supply an overview comment for all source files.
- Place it in a file called overview.html, located in the parent directory that contains all the source files.
- All text between the tags <body>...</body> is extracted.
- This comment is displayed when the user selects “Overview” from the navigation bar.

7. Comment Extraction

- docDirectory is the name of the directory where you want the HTML files to go.
- Follow these steps:

1. Change to the directory that contains the source files you want to document.

- If you have nested packages to document, such as com.horstmann.corejava, you must be working in the directory that contains the subdirectory com. (This is the directory that contains the overview.html file, if you supplied one.)

7. Comment Extraction

- 2. Run the command
- ➢ javadoc -d docDirectory nameOfPackage
- for a single package. Or, run
- ➢ javadoc -d docDirectory nameOfPackage1 nameOfPackage2...
- to document multiple packages.
- If your files are in the default package, run instead
- ➢ javadoc -d docDirectory *.java
- If you omit the -d docDirectory option, the HTML files are extracted to the current directory. That can get messy, so it is not recommended

7. Comment Extraction

- The javadoc program can be fine-tuned by numerous command-line options.
- For example, you can use the -author and -version options to include the @author and @version tags in the documentation. (By default, they are omitted.)
- Another useful option is -link, to include hyperlinks to standard classes. For example, if you use the command

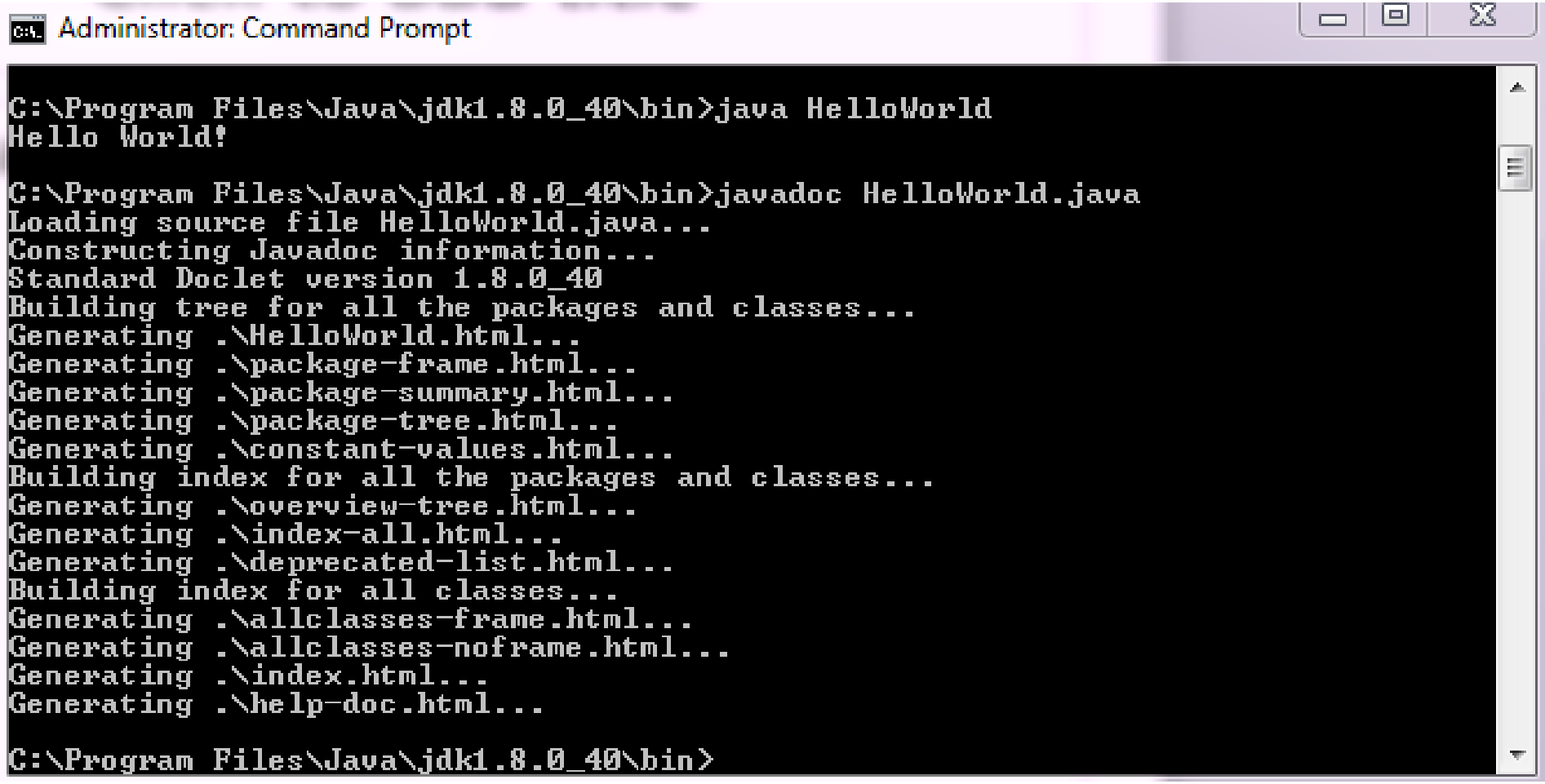
➢ javadoc -link http://docs.oracle.com/javase/7/docs/api *.java

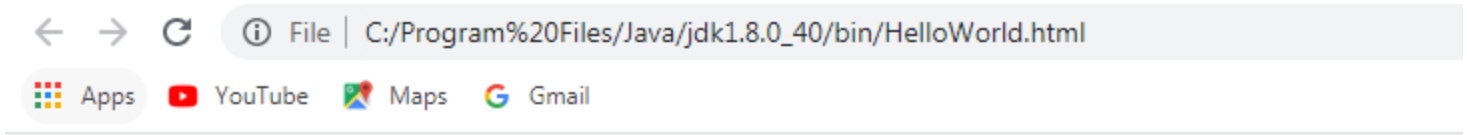
- all standard library classes are automatically linked to the documentation on the Oracle web site.
- If you use the -linksource option, each source file is converted to HTML (without color coding, but with line numbers), and each class and method name turns into a hyperlink to the source.

Example

```
/**
*<h1>Hai II CSE B students!</h1>
* The HelloWorld program implements an application that
* simply displays "Hello World!" to the standard output.
*
* @author  Bamila
* @version 1.0
* @since   5-9-2020
*/
public class HelloWorld {

   public static void main(String[] args) {
      // Prints Hello, World! on standard output.
      System.out.println("Hello World!");
   }
}
```


Administrator: Command Prompt
C:\Program Files\Java\jdk1.8.0_40\bin>java HelloWorld
Hello World!
C:\Program Files\Java\jdk1.8.0_40\bin>javadoc HelloWorld.java
Loading source file HelloWorld.java...
Constructing Javadoc information...
Standard Doclet version 1.8.0_40
Building tree for all the packages and classes...
Generating .\HelloWorld.html...
Generating .\package-frame.html...
Generating .\package-summary.html...
Generating .\package-tree.html...
Generating .\constant-values.html...
Building index for all the packages and classes...
Generating .\overview-tree.html...
Generating .\index-all.html...
Generating .\deprecated-list.html...
Building index for all classes...
Generating .\allclasses-frame.html...
Generating .\allclasses-noframe.html...
Generating .\index.html...
Generating .\help-doc.html...
C:\Program Files\Java\jdk1.8.0_40\bin>



Class HelloWorld

java.lang.Object
    HelloWorld

```
public class HelloWorld
extends java.lang.Object
```

Hai II CSE B students!

The HelloWorld program implements an application that simply displays "Hello World!" to the standard output.

Since:

5-9-2020

Constructor Summary

Constructors
Constructor and Description
`HelloWorld()`

Method Summary

All Methods	Static Methods	Concrete Methods
Modifier and Type	Method and Description	